BLOCKCHAIN IMPLEMENTATION FOR AGRICULTURAL FOOD SUPPLY CHAIN USING HYPERLEDGER FABRIC

ASHRAF ALI RAKIB C213090

MIRAJ MAHMUD C213062

MD ABDULLAH C213042

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG (IIUC)

BLOCKCHAIN IMPLEMENTION FOR AGRICULTURAL FOOD SUPPLY CGAIN USING HYPERLEDGER FABRIC

ASHRAF ALI RAKIB C213090
MIRAJ MAHMUD C213062
MD ABDULLAH C213042

THESIS SUBMITTED IN FULFILMENT FOR THE DEGREE OF
B. SC. IN COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CSE)
INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG (IIUC)
CHITTAGONG, BANGLADESH

SPRING 2025

INTERNATIONAL ISLAMIC UNIVERSITY
CHITTAGONG (IIUC)

DECLARATION

We confirm the following assertions in relation to our thesis:

1. As part of our undergraduate degree program at International Islamic University Chittagong, we successfully completed the thesis.

2. No previously published or unattributed third-party content is included in the thesis work.

3. No other university or institution has previously accepted the thesis for consideration for any other degree or diploma.

4. Every important source that contributed to the thesis has been appropriately acknowledged.

29 October 2025

ASHRAF ALI RAKIB C213090

MIRAJ MAHMUD C213062

MD ABDULLAH C213042

SUPERVISOR'S DECLARATION

I thus certify that, after reviewing the thesis, I have found it to be of adequate quality and scope, supporting the awarding of a Bachelor of Science in Computer Science and Engineering.

29 October 2025

ABDULLAHIL KAFI
Assistant Professor
Computer Science and Engineering
International Islamic University
Chittagong

DECLARATION OF THESIS / PROJECT REPORT AND COPYRIGHT

THESIS / PROJECT REPORT TITLE: BLOCKCHAIN IMPLEMENTATION FOR AGRICULTURAL FOOD SUPPLY CHAIN USING HYPERLEDGER FABRIC

AUTHORS:

SL NO.	AUTHOR'S NAME	STUDENT ID	SIGNATURE
1			
2			
3			
Name OF SUPERVISOR: SIGNATURE OF SUPERVISOR:			

I/we declare

1. My/ Our thesis to be published as online open access (full text) at IIUC database or archive.

2. Dept. of CSE, IIUC reserves the right as follows:

i. The thesis is the property of Dept. of CSE, IIUC

ii.The Library of IIUC has the right to make copies for the purpose of research only.

iii. The Library has the right to make copies of the thesis for academic exchange.

ACKNOWLEDGEMENT

Above all, we thank the Almighty Allah for His generosity, which allowed us to finish our thesis in spite of many challenges. Second, we would like to express our gratitude to Abdullahil Kafi, our supervisor, for his unwavering support and direction since the beginning of our study.

ABSTRACT

The agricultural food supply chain in Bangladesh is posing a great challenge with majority of them being that there is a major lack of transparency in the system, inefficiency in an agricultural system coupled with high level of distrust towards consumers. It is these inadequacies which tend to be the source of food quality and other disputes that bring economic harm to our agrarian society. To address this complex issues, it is worth implementing an efficient system that is able to ensure the provenance and integrity of food items used at the cultivated level to the end table of the consumer. In this dissertation, the system of agricultural supply chain being traced is decentralized and operates with the Hyperledger Fabric as a permissioned blockchain system. The primary objective is the establishment of a safe, irreversible and transparent registry of agricultural products and their lifecycle in order to increase food safety and provide interested parties with a new level of control.This methodological design entails the setting up and execution of a Hyperledger Fabric multi-organization network, which embraces and enforces the three primary players: Farmers, Retailers and Consumers. The Raft algorithm consensus deals with the network and has been selected due to its crash-fault tolerance and capability to work efficiently in a permissioned environment. Business logic is coded in an all-encompassing smart contract written in Go. ProductID, ProductLocation, and Quality are some of the attributes found in the digital asset of a Product which is controlled by this chaincode.A very restrictive, role-based transaction workflow of creating a Product by Farmers, updating Product status by Retailers and a Product purchased by Consumers governs the system. Such transactions leave a permanent score on a record of historical registry and this creates a complete audit trail which is verifiable.They showed that the network had been instantiated appropriately in a testbed and that the key functionality of the network had been put to test using a series of simulated transactions, which followed the route between farm and consumer. The empirical results prove that the platform can offer a tamper-proof report of devoted agricultural products. This is because the role permissions worked and the system provided integrity of data throughout the process of supply chain.Finally, the project suggests a prospective and efficient blockchain infrastructure, which can develop traceability and accountability in the agricultural industry. It is an effective option to control fraud, streamline the working process, and restore the trust of all the participants in the food chain.

Keywords: Blockchain, Hyperledger Fabric, Agricultural Supply Chain, Food Traceability, Smart Contracts, Raft Consensus, Data Integrity, Consumer Trust.

TABLE OF CONTENTS

DECLARATION			**i**
SUPERVISOR'S DECLARATION			**ii**
DECLARATION OF THESIS / PROJECT REPORT AND COPYRIGHT			**iii**
ACKNOWLEDGEMENT			**iv**
ABSTRACT			**v**
TABLE OF CONTENTS			**vi**
LIST OF TABLES			**xi**
LIST OF ILLUSTRATIONS			**xii**
LIST OF ABBREVIATIONS			**xiii**
CHAPTER I	**INTRODUCTION**		**1**
1.1	**RESEARCH BACKGROUND**	**1**	
1.1.1	**KEY POINTS**	**2**	
1.2	**PROBLEM STATEMENT**	**2**	
1.3	**MOTIVATION**	**3**	
1.4	**OBJECTIVE OF RESEARCH**	**3**	
1.5	**CONTRIBUTION OF THE RESEARCH**	**4**	
1.6	**ORGANIZATION OF THE THESIS**	**4**	
CHAPTER II	**LITERATURE REVIEW**		**5**

2.1	**INTRODUCTION**	**5**
2.2	**SCOPE OF RESEARCH**	**5**
2.3	**THE EVOLUTION OF SUPPLY CHAIN TECHNOLOGY**	**6**
2.4	**BLOCKCHAIN TECHNOLOGY FUNDAMENTALS**	**6**
2.5	**TYPES OF BLOCKCHAIN NETWORKS**	**7**
2.6	**A CLOSER LOOK AT HYPERLEDGER FABRIC**	**7**
2.7	**SMART CONTRACTS AND CONSENSUS**	**8**
2.8	**THE RAFT CONSENSUS ALGORITHM**	**8**
2.9	**THE REASONS FOR SELECTING HYPERLEDGER FABRIC**	**8**
2.10	**LITERATURE REVIEW**	**12**
2.11	**Research Gap and Conclusion**	**12**
2.12	**SUMMARY**	**13**

CHAPTER III METHODOLOGY 14

3.1 INTRODUCTION 14

3.2 TOOLS USED 14

3.3 OTHER NECESSARY TECHNOLOGIES 15

3.3.1 HASHING ALGORITHM (SHA-256) 16

3.3.2 CONSENSUS ALGORITHM (RAFT) 16

3.3.3 ADVANTAGES OF RAFT FOR OUR SYSTEM 18

3.3.4 IMPLEMENTATION OF RAFT IN HYPERLEDGER FABRIC 19

3.3.5 HASHING ALGORITHM 20

3.3.6 CONSENUS ALGORITHM 21

3.3.7 CERTIFICATE AUTHORITY(CA) 22

3.4 NETWORK DEPLOYMENT STEPS 23

3.5 NETWORK 24

3.5.1 ARCHITECTURE OF NETWORK 24

3.5.2 WORKFLOW OF NETWORK 26

3.6 STEP-BY-STEP PROCESS OF THE SMART CONTRACT 27

3.6.1 TEST CASES FOR ACCESS CONTROL 31

3.7 INTEGRATING HYPERLEDGER EXPLORER FOR VISUALIZATION 33

3.8 SUMMARY 38

CHAPTER IV RESULTS AND DISCUSSION 39

4.1 INTRODUCTION 39

4.2 SYSTEM DEPLOYMENT AND LIFECYCLE MANAGEMENT 39

4.3 TRANSACTION TESTING AND VALIDATION OF BUSINESS RULES 40

4.4 DISCUSSION OF RESULTS 41

4.4.1 VALIDATION OF THE TECHNICAL IMPLEMENTATION 41

4.4.2 ENFORCEMENT OF BUSINESS LOGIC AND TRUST 41

4.4.3 CONTRIBUTION TO TRACEABILITY AND TRANSPARENCY 41

4.4.4 LIMITATIONS AND FUTURE WORK 41

4.5 COMPARATIVE ANALYSIS 42

4.5.1 KEY FEATURE COMPARISON 43

4.5.1.1 CORE APPLICATION 43

4.5.1.2 SUPPLY CHAIN MODEL 43

4.5.1.3 KEY INNOVATION 43

4.5.1.4 SMART CONTRACT LANGUAGE 44

4.5.1.5 PERFORMANCE ANALYSIS 44

4.5.1.6 REPRODUCIBILITY 44

4.5.1.7 HYPERLEDGER EXPLORER INTEGRATION 44

4.5.1.8 CANAL LOCK(STRICT SEQUENTIAL FLOW) 45

4.5.1.9 SUMMARY OF FINDINGS 45

4.6 PERFORMANCE ANALYSIS 46

4.6.1 THROUGHPUT (TPS) 46

4.6.2	**LATENCY**	**47**
4.7	**SUMMARY**	**47**
CHAPTER V	**Conclusion and Future Works**	**48**
5.1	**SUMMARY OF THE FINDINGS**	**48**
5.2	**LIMITATIONS AND FUTURE WORK**	**49**
5.3	**CONTRIBUTION**	**50**
References		**51**
Appendix A	**CHAINCODE (AGRIFOOD.GO)**	**53**
Appendix B	**NETWORK SETUP AND CLI COMMANDS**	**57**
Appendix C	**HTPERLRDGER EXPLORER SETUP**	**64**
Appendix D	**COMPLEX ENGINEERING PROBLEM**	**67**

LIST OF TABLES

Table No.	Page
Table 1 : Related Works	10
Table 2 : Tools used	15
Table 3 : Test Case Results	40
Table 4 : Comparison with existing works	42

LIST OF ILLUSTRATIONS

Figure No. **Page**

Figure 1 Importance of Hyperledger Fabric 9

Figure 2 : Raft Consensus Algorithm mechanism 18

Figure 3 : Hashing algorithm 20

Figure 4 : Orderer Consensus 21

Figure 5 : Channel Consensus 22

Figure 6 : Network Architecture 25

Figure 7 : Agri-Food Supply Chain Workflow **Error! Bookmark not defined.**

Figure 8 : Product Structure Code Snippet 27

Figure 9 : Adding a New Product Code Snippet 28

Figure 10 : Adding a New Product CLI Command 29

Figure 11 : Update Product Status(Farmer->Retailer) Code Snippet 29

Figure 12 : Update Product Status (Farmer->Retailer) CLI Command 30

Figure 13 : Purchase Product (Retailer->Customer) Code Snippet 30

Figure 14 : Purchase Product (Retailer->Customer) CLI Command 31

Figure 15 : Change in docker-compose.yaml 35

Figure 16 : Explorer Dashboard 37

Figure 17 : A Block Details 37

LIST OF ABBREVIATIONS

The following list explains some of the symbols and abbreviations that will be used throughout the rest of the document.

HLF: Hyperledger Fabric

CA: Certificate Authority

DLT: Distributed Ledger Technology

MSP: Membership Service Provider

TLS: Transport Layer Security

ORG: Organization

NGOs: Non-Governmental Organizations

CHAPTER I INTRODUCTION

Digital technologies like the blockchain, Big Data Analytics, and Internet of Things (IoT) are changing the way food gets from farms to stores, making it safer, more open, and more efficient. Blockchain's decentralized, unchangeable, and open ledger keeps track of every transaction on the network, which builds confidence between growers, processors, distributors, retailers, and customers. When combined with IoT sensors, it can collect real-time information on how things are stored, how they are transported, and how they are handled.[20]This makes a record that can be checked and can't be changed from farm to table. The agricultural food supply chain has problems that never go away, such as product fraud, contamination, mislabeling, and inadequate traceability. These problems make people less likely to trust the food and put their safety at risk. Conventional centralized systems frequently lack the capability for real-time, comprehensive monitoring, complicating the identification of issue origins during recalls or contamination incidents. This thesis suggests using the Hyperledger Fabric framework, which is a permissioned blockchain platform, to develop a strong traceability solution. The system will use smart contracts, digital certification, to keep track of quality metrics, speed up recall processes, and encourage sustainable behaviors. This will improve both consumer trust and the robustness of the supply chain.[1] [13]

1.1 RESEARCH BACKGROUND

The agricultural supply chain is very important for the economies of emerging countries like Bangladesh. But traditional management methods that use paper records or separate digital databases often get in the way of its full potential. These old ways of doing things aren't clear or easy to follow, which makes them vulnerable to fraud, arguments over where a product came from, and less trust from customers. This not only hurts farmers' businesses, but it also puts the public at danger for food safety because there aren't any dependable ways to trace food. Blockchain technology is a game-changer because it lets you create a digital ledger that is decentralized, immutable and open to everyone. It is especially good for supply chain management since it has a built-in feature that makes it impossible to change the audit trail. This is important for checking the source of agricultural products from farm to consumer. Hyperledger Fabric was chosen as the main framework for this study. It is a permissioned blockchain technology that is made for business settings like food supply chains, where everyone knows who they are and following the rules is very important. Smart contracts (chaincode) are what make the system's business logic work. They enforce rules for transactions, including producers making products, intermediates transferring them, and retailers selling them. The Raft consensus algorithm is used to keep data consistent and make sure the network can handle errors. Raft offers efficient and crash fault-tolerant consensus without the expensive expense of proof-of-work systems.[4] [5]

1.1.1 KEY POINTS:

Problem: Traditional agricultural supply chains aren't clear, which leads to fraud, waste, and low consumer trust.

Solution: A decentralized system based on blockchain technology that makes a safe and immutable record of a product's journey.

Framework: Hyperledger Fabric is employed because it is permissioned, which makes it good for a regulated business network with known participants (Farmers, Retailers, and Consumers).

Automation: Smart contracts (chaincode) automate and enforce the rules for transactions, making sure that permissions based on roles are followed.

Reliability: The Raft consensus algorithm is a reliable and effective approach to make sure that all participants have the same ledger.

1.2 PROBLEM STATEMENT

There is a big knowledge gap between the main stakeholders in Bangladesh's agricultural supply chain. This gap makes things less clear and traceable, which leads to a cycle of problems that affect everyone involved, from producers to end customers. Farmers have a harder time negotiating when they can't prove where their food came from or how good it is. Intermediaries might challenge claims about quality or where something came from, which can lead to unfair prices and lower income. Farmers can't get into premium markets like organic or export-grade segments because they don't have a reliable way to verify that their products are of good quality. Instead, they have to accept lower market rates. Sourcing is risky for retailers. Retailers risk losing money if they buy items that are not up to par, and they risk damaging their brand if dangerous or fake products reach customers. This is because they have few ways to check suppliers' statements about where the goods came from, when they were harvested, or how they were handled. Without a reliable way to check, retailers are completely responsible for quality assurance, which makes the process expensive and time-consuming. The main problem for customers is that they don't trust the company. There is no easy method to check if the food you bought is real. People are worried about a lot of things, such as using pesticides that aren't allowed, identifying the wrong place of origin (for example, marketing product from one area as if it were from a better-known area), and not storing things properly. This lack of information makes it harder to make smart choices and raises the risk of health problems. Traditional record-keeping, whether on paper or in a central digital system, has not been able to fix these challenges. These kinds of systems are broken, full of mistakes, and easy to change because there is no one source of truth that everyone trusts. This study addresses the pressing requirement for a cohesive, secure, and transparent platform that can permanently monitor agricultural products from their source on the farm to their ultimate sale, thereby reinstating integrity and confidence throughout the whole supply chain.[4]

1.3 MOTIVATION

This project is motivated by the conviction that technology may resolve significant societal issues inside Bangladesh's agricultural supply chain, where inequality, risk, and mistrust adversely affect farmers, retailers, and consumers. The main goal is to give farmers more power by letting them record and prove the quality and source of their goods. This will help them get fair pricing and get into high-end markets. It also wants to give retailers a reliable way to check things out so they can lower the chances of getting bad products and keep their businesses safe from financial and reputational damage. The goal for consumers is to reestablish their faith in the safety and authenticity of food by giving them direct access to clear product histories. The goal is to use Hyperledger Fabric to make a trustworthy, shared ledger that gets beyond the problems of paper-based and centralized systems. This is possible since blockchain is unchangeable, open, and secure. Another reason is to meet regulatory and export traceability criteria, which will make Bangladesh more competitive in global markets. In the end, the goal of the project is to create a supply chain that is fair, trustworthy, and good for the environment for everyone involved.

1.4 OBJECTIVE OF RESEARCH

The main goal of this project is to create, test, and evaluate a decentralized application that tackles the important problems of trust and traceability in Bangladesh's agricultural food supply chain. The following specific targets are being worked toward to reach this main goal:

Design and Deploy a Permissioned Blockchain Network: Create a strong Hyperledger Fabric network for the agricultural supply chain that includes three different groups: Farmers, Retailers and Consumers. The Raft consensus algorithm will be used by the network to make sure that it is reliable, secure, and able to handle errors.[7] [19]

Implement a Smart Contract for End-to-End Traceability : Create a full smart contract (chaincode) in Go to simulate the whole life cycle of farm products. This includes setting up a thorough Product asset structure and adding important services like CreateProduct, UpdateProductStatus, and PurchaseProduct to make an immutable, verifiable record of each transaction.

Enforce Role-Based Access Control : Add rigorous permission limits to the smart contract so that only verified members of the right organization may carry out role-specific tasks. This will improve security and keep transactions honest.

1.5 CONTRIBUTION OF THE RESEARCH

This research focuses on developing innovative techniques to establish a transparent and immutable agricultural supply chain using permissioned blockchain technology. The specific objectives are:

- **Design and deploy** a permissioned blockchain network using Hyperledger Fabric, tailored to the operational needs of farmers, retailers, and consumers.
- **Implement** a custom smart contract (chaincode) that ensures end-to-end traceability of agricultural products, from farm production to final sale.
- **Enforce** strict role-based access control within the smart contract to safeguard the security, authenticity, and integrity of all transactions.

1.6 ORGANIZATION OF THE THESIS

This thesis is organized into five chapters, as outlined below:

- **Chapter 1: Introduction** – Outlines the foundation of the study, presenting the problem statement, research motivation, and objectives.
- **Chapter 2: Literature Review** – Examines the core technologies utilized, including blockchain, Hyperledger Fabric, and the Raft consensus algorithm, and reviews prior research on blockchain applications in supply chain management.
- **Chapter 3: Methodology** – Describes the design and implementation of the proposed system, detailing the network architecture, setup of the Hyperledger Fabric test network, and the development of the smart contract logic and functions.
- **Chapter 4: Results and Discussion** – Presents the outcomes of the system validation phase, demonstrating the execution of chaincode functions, transaction simulations, and evaluating the system's effectiveness in establishing a transparent audit trail.
- **Chapter 5: Conclusion** – Summarizes the research findings, discusses the limitations of the work and suggests directions for future research and potential system enhancements.

CHAPTER II : LITERATURE REVIEW

2.1 INTRODUCTION

The modern agricultural food supply chain is an important part of the world economy, but it has problems and weaknesses that have big impacts. Concerns include food fraud, which costs the worldwide sector billions of dollars each year, contamination events that result in costly and lengthy recalls, and a general lack of transparency. These things not only hurt the economy, but they also put public health at risk and damage customer trust. One of the main reasons for these problems is because the structure of record-keeping is often broken and hard to understand. Data is often stored on paper or in separate, centralized systems in many different companies. In this situation, it's almost impossible to make a single, verifiable, and up-to-date history of a food product's trip from farm to fork. Industry 4.0 has brought about a new set of technologies that could totally change traditional industries. Blockchain technology is one of them that has become an important layer of trust that might totally change how supply chains are managed. Blockchain provides a distinctive solution to the ongoing challenges of data fragmentation and distrust among supply chain participants by presenting a decentralized, transparent, and immutable ledger. This thesis addresses these critical vulnerabilities by proposing and implementing a blockchain-based system for an agricultural food supply chain. The research examines the potential of a decentralized ledger to enhance transparency and trustworthiness within the ecosystem for all participants, ranging from the first farmer to the final consumer. This chapter will set the intellectual and technical groundwork for the effort by looking at the body of existing research, defining crucial concepts, and explaining why Hyperledger Fabric is the best technology for the proposed solution.

2.2 SCOPE OF RESEARCH

This study focuses on the application of permissioned blockchain technology within the agrifood sector. We will focus on the Hyperledger Fabric framework because it is designed for enterprise-level consortia with known and verified participants.

In Scope: This review will look at how Go smart contracts, the Raft consensus mechanism, and Hyperledger Fabric can be used. In a food traceability system, the primary interactions between a farmer (Org1), a retailer (Org2), and a customer (Org3) are simulated in the real world.[15]

Out of Scope: This study does not cover public (permissionless) blockchains such as Ethereum or Bitcoin, along with their associated currency, nor does it consider computationally intensive consensus mechanisms like Proof-of-Work. The direct integration of hardware, including Internet of Things (IoT) sensors, is also viewed as

outside the current scope, although it will be looked at as a prospective topic for future development. [3]

2.3 THE EVOLUTION OF SUPPLY CHAIN TECHNOLOGY

To fully understand the importance of blockchain, it helps to know the history of supply chain technology. In the beginning, supply chains used paper-based systems, with real bills of lading, purchase orders, and quality certificates. This process was very slow, easy to cheat, and easy to make mistakes. The digital revolution led to the creation of Enterprise Resource Planning (ERP) systems, which made businesses much more efficient. These unified databases made it possible for one business to handle its finances, logistics, and inventory all at once. But they didn't talk about how different supply chain businesses share data. Each stakeholder had their own private ERP, which led to data silos and a lack of a consistent, reliable view of the whole chain. To connect these different systems, a new type of technology was needed: a decentralized trust layer. Blockchain is what this is all about.[8]

2.4 BLOCKCHAIN TECHNOLOGY FUNDAMENTALS

A Comprehensive Overview of Blockchain Ideas: Our application requires a strong understanding of the foundations of blockchain technology.
Distributed Ledger Technology (DLT): A blockchain is fundamentally a sort of distributed ledger technology. A distributed ledger, as opposed to a traditional centralized database owned by a single corporation, is a database that is shared, replicated, and synchronized by network participants. It functions as a shared "digital notebook" in which users can collaborate to capture transactions. Shared visibility creates confidence by ensuring that each participant has an identical copy of the ledger, eliminating any single point of failure.

Cryptographic Hashing: Cryptographic hashing is critical for a blockchain's security and immutability. A hash function, such as SHA-256, can accept any size input and return a fixed-length string of characters. This result, or "hash," functions as the data's unique digital fingerprint. Any manipulation is immediately apparent because even the smallest change to the input data results in an altogether different hash.

Block Structure and Immutability: A blockchain is composed of blocks, each of which contains a timestamp, a list of transactions, most importantly, the hash of the block preceding it. The hash from the previous block is integrated into the current block, resulting in a secure and unbreakable chain. If an attacker modifies a transaction in a prior block, the hash of that block changes. If this update resulted in a mismatch with the hash recorded in the next block, the entire chain would break. The ledger's chain-like structure makes it immutable, which means it cannot be altered or changed.

2.5 TYPES OF BLOCKCHAIN NETWORKS

Private vs. Public Blockchains: Blockchain networks are classified as either private (permitted) or public (permissionless). Knowing the distinction is critical for understanding why one is more suited for corporate supply chains**.**

Public (Permissionless) Blockchains: These networks, which include Ethereum and Bitcoin, are open to anybody. Anyone can join the network, view all transactions that have ever occurred, and participate in the consensus process. Typically, computationally intensive "mining" techniques like as Proof-of-Work are used to secure them. Despite their unparalleled decentralization, they have significant drawbacks for corporate applications, including as poor transaction speeds, high latency, high energy costs, and no data privacy at all**.** [10]

Private (Permissioned) Blockchains: In contrast, Hyperledger Fabric and other permissioned blockchains operate on a "invite-only" basis. Every member of the network is recognized, known and has been granted specific rights and authorizations. This architecture is meant for corporate consortiums that require a shared, verified ledger to speed up procedures but already have a high level of confidence. High transaction performance, data confidentiality, and lower operating costs are among the many benefits. The permissioned approach is the only viable option for a regulated area, such as the agricultural food supply chain, where stakeholders are well-known (farmers, retailers, etc.) and data may be financially sensitive.

2.6 A CLOSER LOOK AT HYPERLEDGER FABRIC

Hyperledger Fabric Architecture: The Hyperledger Fabric architecture is exceptionally versatile and adaptable, making it appropriate for a wide range of industry use cases. The primary aspects supporting our project's network are as follows:

Peers: These are the network's core nodes. They execute smart contracts, also known as chaincode in Fabric, and store copies of the ledger. In our model, the Farmer and the Retailer are instances of participating organizations that own and govern their peers.

Orderers: These nodes make up the "ordering service". Their primary responsibility is to obtain an agreement on the sequence of transactions, which they will then organize into blocks and disseminate to their peers. Our ordering method employs the Raft consensus mechanism.

Certificate Authorities (CA): The certificate authority serves as the network's trust anchor. It issues digital certificates that serve as verifiable identities to all participants (users, peers, and orderers). A major component of the permissioned paradigm is the CA's assurance that each entity on the network is who they claim to be.
Channels: Channels are a useful data privacy feature. They serve as unique "sub-networks" within the larger blockchain network, allowing transactions by a certain set

of users in complete secrecy and seclusion. This indicates that participants in one channel cannot see data in another**.**

Membership Service Provider (MSP): An MSP is a component that sets up an organization's identity validation procedures and laws. Each network organization, such as our Farmer (Org1MSP) has its own MSP that manages members' rights and certificates. [6]

2.7 SMART CONTRACTS AND CONSENSUS

Smart Contract (Chaincode) Function: The business logic layer of a blockchain is based on smart contracts. These are known as chaincode in Hyperledger Fabric. They are self-executing programs in which the distributed ledger stores and duplicates the code containing the terms and rules of an agreement. Chaincode increases productivity while lowering the risk of malicious action or human error.The Go chaincode that was utilized to implement our project defines the entire set of regulations for the agricultural supply chain. It enforces stakeholder roles programmatically.

The logic in the CreateProduct function authenticates the caller's digital identity. It will only run if the caller is a Farmer organization member (Org1MSP). Similarly, there are checks in the UpdateProductStatus and PurchaseProduct procedures to ensure that only the Retailer (Org2MSP) and Customer (Org3MSP) can use them. One of the system's primary benefits is that it automatically enforces corporate standards.[6]

2.8 THE RAFT CONSENSUS ALGORITHM

Consensus is the way that the nodes in a distributed network agree on the present state of the ledger. Our solution uses the Raft consensus process, which is part of Fabric's ordering service. Raft uses a "leader and follower" model, where one orderer node is chosen to lead a certain channel.The way the procedure works is that client apps send their transaction proposals to other clients for approval. After that, the leader node of the ordering service gets the transactions that have been approved. The leader has complete control over the sequencing of these transactions and how they are grouped into a new block. Then, this block is copied to the follower orderer nodes. After most of the people who ordered it say they got the block, it is transmitted to everyone on the channel to be added to their copy of the ledger and is considered final. This leader-based method delivers the low latency and high throughput that a real-world supply chain application needs. It is also very effective and can handle crashes (the network can keep working even if some nodes fail). [11]

2.9 THE REASONS FOR SELECTING HYPERLEDGER FABRIC

After comparing its features to the specific needs of an agricultural food supply chain, Hyperledger Fabric was picked deliberately. Its permissioned architecture is very

important for a commercial consortium of well-known enterprises that need to be held accountable. In addition, its capability for private channels is an important way to protect data because it lets certain parties share sensitive economic information without exposing the whole network. Fabric is made to work at the enterprise level above everything else. As shown in the linked performance benchmarks, a Fabric network implementing Raft consensus consistently outperforms other blockchain platforms in parameters that are important for a high-volume supply chain.

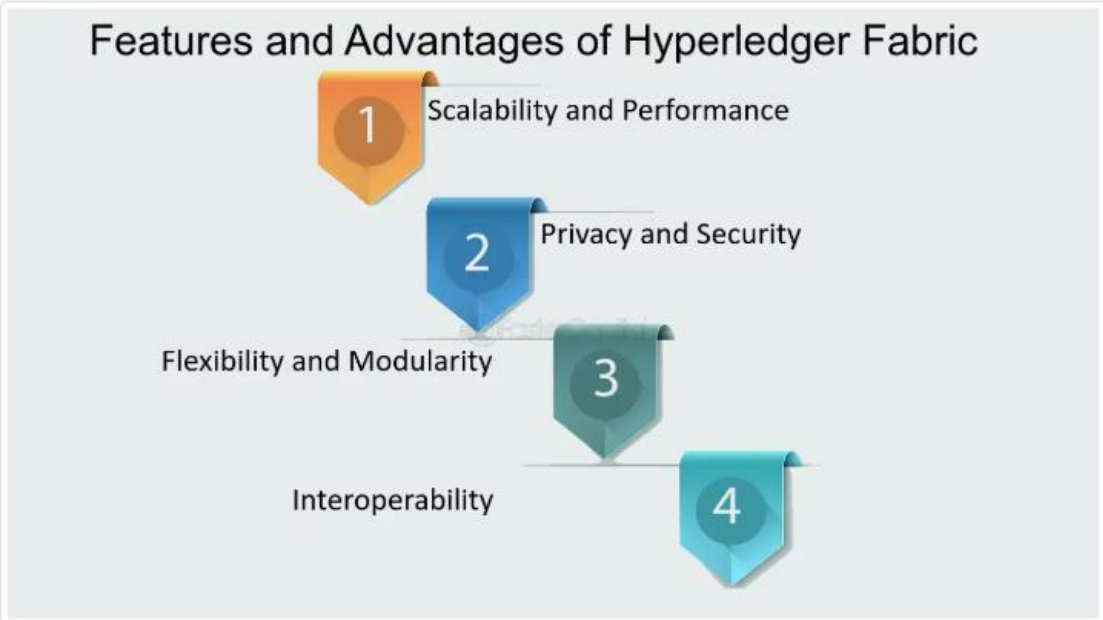

Figure 1: Importance of Hyperledger Fabric

Criteria	Hyperledger Fabric (Raft)	Hyperledger Sawtooth (PoET)	Ethereum (PoW)
Network Type	Permissioned – access restricted to authorized participants	Permissioned – access limited to approved members	Permissionless – open to anyone globally
Privacy Level	High – secured via channel architecture for private transactions	Low – minimal privacy controls	Low (Pseudonymous) – identities hidden but transactions public
Transaction Throughput (TPS)	High – optimized for enterprise use	Medium – suitable for moderate workloads	Low – limited by consensus mechanism
Latency	Low – near real-time transaction finality	Medium – moderate confirmation times	High – slow block confirmation times
Consensus Mechanism	Raft (CFT) – crash fault-tolerant	PoET (BFT-like) – proof-of-elapsed-time consensus	Proof-of-Work (Probabilistic) –

	leader-follower model		mining-based validation
Governance Model	Defined Consortium – governed by participating organizations	Defined Consortium – managed by approved entities	Decentralized – governed by community consensus

This comparison clearly indicates that for a private, high-performance, and scalable supply chain solution, Hyperledger Fabric is the best technological choice.

2.10 LITERATURE REVIEW

Table 1 : Related Works

Year	Title	Author(s)	Approach & Technology	Strengths	Limitations
2024	Transforming Agricultural Supply Chains: Leveraging Blockchain-Enabled Java Smart Contracts and IoT Integration	A. El Mane, K. Tatane, Y. Chihab [16]	Hyperledger Fabric with Java smart contracts and IoT sensors for real-time environmental data capture across farm networks.	Combines IoT with blockchain for automated on-chain data entry; practical for multi-farm agricultural networks.	Uses Java smart contracts, which are less efficient than Go; no quantitative performance benchmarking reported.
2022	Blockchain implementation for agricultural food supply chain using Hyperledger Fabric	Suganya Balasubramanian, I.S. Akila	Decentralized Application (DApp) using Hyperledger Fabric for agri-food supply chain traceability.	Implemented a prototype system and validated it with smart contracts. Integrated QR codes for consumer verification.	The DApp uses MongoDB for data storage, which can introduce a single point of failure if not properly managed.
2022	A Blockchain-Based System for Agri-Food Supply Chain Traceability Management	A. Marchese, O. Tomarchio [15]	Hyperledger Fabric-based system for end-to-end agri-food supply chain traceability management.	Demonstrates viability of HLF for food-specific use cases; provides end-to-end traceability design.	No quantitative performance benchmarking reported; lacks role-based access

					control enforcement.
2025	Application of Hyperledger Blockchain Technology to Logistics Supply Chain with IoT	Ahmet Sayar, Muhammet Cüneyd Kurtbaş, Çağlayan Sancaktar, Rana Dudu Kabak, Şükrü Çakmak	Integration of Hyperledger Fabric with IoT devices using the MQTT protocol for real-time logistics tracking.	Addresses the security vulnerabilities of IoT by using a blockchain-based system. Provides a strong foundation for real-time, automated tracking.	The work is based on a simulation environment and does not provide real-world performance metrics for a large-scale network.
2022	Empirical Performance Analysis of Hyperledger LTS for Small and Medium Enterprises	D. Khan, L. T. Jung, M. A. Hashmani, M. K. Cheong [19]	Empirical benchmarking of Hyperledger Fabric LTS for SME use cases; tests throughput and latency at varying node counts.	Provides empirical TPS and latency data; demonstrates stable throughput with proportional latency scaling.	Focused on general SME performance rather than agri-food supply chain; no access control validation.
2024	Smart Agriculture Assurance: IoT and Blockchain for Trusted Sustainable Produce	H. R. Hasan et al. [14]	Combines IoT sensors with blockchain for smart agriculture assurance and trusted produce traceability.	Addresses real-world agricultural trust issues; integrates IoT with blockchain for automated monitoring.	No fully reproducible implementation provided; performance metrics not reported.
2019	The Rise of Blockchain Technology in Agriculture and Food Supply Chains	A. Kamilaris, A. Fonts, F. X. Prenafeta-Boldu [13]	Comprehensive survey of blockchain applications in agriculture and food supply chains; reviews existing	Broad overview of the field; identifies key use cases and challenges; foundational reference for agricultural blockchain research.	Survey paper without a concrete implementation; does not provide performance benchmarking data.

			implementations and trends.		

2.11 RESEARCH GAP AND CONCLUSION

The review of the existing literature has revealed a significant research deficit. There is a lack of functional, comprehensive implementations that simulate the entire product lifecycle from the farm to a verifiable consumer purchase. This is despite numerous studies proposing theoretical frameworks for agrifood blockchains and others demonstrating partial implementations, such as focusing solely on the producer-distributor relationship.Additionally, many of the existing prototypes in the literature do not possess thorough, reproducible deployment and testing protocols on an operational multi-organization network. Consequently, other researchers encounter difficulties in augmenting or substantiating their results. This thesis argues that a full plan is needed that not only shows how the model will work but also lists all the steps that need to be taken to put it into action and test it.[12] [14]

2.12 SUMMARY

This literature review has shown that blockchain technology could be a good way to solve the big problems in the agricultural food supply chain, such as the lack of trust, transparency, and traceability. Hyperledger Fabric is the greatest technological choice for a corporate consortium since it has greater security, privacy, and performance features than other blockchain types and frameworks. This thesis builds on the established ideas in the literature in order to come up with and carry out a feasible solution. This study fills a major gap in the body of previous academic research by providing a credible and verifiable way for end-to-end food traceability through the use of specifically designed smart contracts that simulate the distinct interactions among farmers, retailers and consumers.

CHAPTER III : METHODOLOGY

3.1 INTRODUCTION

This chapter discusses the work that went into making a blockchain-based agri-food supply chain management system that uses blockchain technology to make the supply chain clear and immutable. In the agri-food sector, a lack of openness can lead to food fraud and public health problems. To reduce these risks, our solution adds a decentralised, tamper-proof ledger that keeps track of the movement and ownership of farm products from farmers to consumers. We look into the technical parts in depth, tell you why we chose them, and show you how they fit into the complete system. We discuss about the blockchain platform being used (Hyperledger Fabric), the synchronisation algorithm (Raft) that makes sure data transactions are safe and quick, and the usage of cryptographic techniques to make sure data is correct and prove who you are. We also talk about how to build and execute smart contracts that enforce business rules and automate tasks like checking for authenticity and keeping track of product quality. This chapter also gives a step-by-step guide to setting up a network, including how to set up blockchain nodes, make secure channels, and install smart contracts. We want to help people understand and copy the technical basis of the system by breaking down each step of the process. We also look at how some groups, like farmers, retailers, and customers, use the blockchain to log and verify transactions.This chapter is divided into sections so that you may learn more about how the system works. The goal is to get a deep understanding of how blockchain technology might work together to stop food fraud and make people more trusting of the agri-food supply chain.

3.2 TOOLS USED

The table below shows the main technology and tools we utilise to build our blockchain-based system for managing the supply chain for agri-food. The chosen tools were the ones that met the system's technical needs in the best way.

Table : Tools used1

Technology	Tools	Description	Used For
Operating System	**Ubuntu**	A Linux-based OS is known for its stability, security, and compatibility with open-source tools.	Development and deployment environment
Platform	**Hyperledger Fabric**	A modular blockchain platform providing enterprise-grade scalability and privacy.	Blockchain network
Container	**Docker, Docker Compose**	Tools for containerization and orchestration, ensuring a consistent and portable environment.	Managing network components as containers
Language	**Go**	A statically typed language known for performance and simplicity.	Writing smart contracts (chaincode)
Database	**LevelDB**	A simple, fast key-value store used as the state database for the Certificate Authority (CA) backend.	CA backend

3.3 OTHER NECESSARY TECHNOLOGIES

In addition to the above core technologies, the following were critical pieces that allowed the system to function, remain secure, and remain efficient:

3.3.1 HASHING ALGORITHM (SHA-256)

Our blockchain-based agri-food supply chain management involves the SHA-256 hashing algorithm integral to it, which provides data integrity and security.

- **Data Integrity and Tamper Detection: Each transaction or block of data is converted to a unique SHA-256 hash.The hash for the transaction data changes entirely if even a single bit of the data is changed, indicating tampering.**
- **Immutability and Trustworthiness:** We allow for a system by which each block of entirely if even a single bit of the data is changed, indicating tampering.Transactions is cryptographically linked to the just preceding block by enabling SHA-256. This 'chain of hashes' ensures that each transaction cannot be tampered with after it's been recorded.
- **Enhanced Security in Product Verification:** In product authentication, SHA-256 makes a hash of every product record stored in the blockchain. If a product goes through the supply chain, its associated hash is re-verified at all checkpoints to ensure that it matches the record.

SHA-256 Integration in the Blockchain System: The SHA-256 cryptographic hash function is integral to our system, providing the foundational layers of data integrity and security. It is utilized at three key stages of the blockchain's operation:

- **Transaction Hashing:** Before any transaction is added to the blockchain, a unique digital fingerprint of that transaction is created using SHA-256. This hash is then stored within the block, ensuring that the integrity of the transaction data cannot be compromised. Any alteration to the transaction would result in a different hash, making the change immediately detectable and invalid.
- **Block Linking:** Each block in the blockchain contains the unique SHA-256 hash of the block that precedes it. This cryptographic link creates a continuous and tamper-evident chain of records. To modify a single block, an attacker would need to re-compute the hashes for all subsequent blocks in the chain, a task that is computationally unfeasible without network-wide consensus. This mechanism is the core of the blockchain's immutability.
- **Product Tracking:** We use SHA-256 to create a unique digital fingerprint for each product record. This hash is generated from key product details such as the batch number, origin, and quality data. Every time a product reaches a new checkpoint in the supply chain, its hash is verified against the original record on the ledger. This multi-step authentication process ensures that no counterfeit or altered products can pass through the system undetected.

3.3.2 CONSENSUS ALGORITHM (RAFT)

A distributed consensus mechanism implemented by multiple nodes working towards reliability and consistency is the **Raft algorithm**. This design is particularly well-suited

for use in private blockchain networks like Hyperledger Fabric where high throughput and fault tolerance requires a solution that doesn't compromise on performance. In our blockchain-based agri-food supply chain system, Raft is used to guarantee that every transaction is traceably recorded and consistently replicated across all nodes despite possible network failures. [11]

Advantages of Raft

1. **High Availability and Efficiency:** Raft is built to handle partial node failures without stalling, allowing the network to handle transactions easily. Unlike computationally expensive Proof of Work (PoW) or Proof of Stake (PoS), Raft is extremely efficient, achieving consensus with minimal resource consumption.
2. **Strong Consistency:** Raft guarantees a strong consistency model, ensuring that all nodes maintain an identical and correct transaction history. This prevents discrepancies in transaction records, which is crucial for an agri-food supply chain.
3. **Deterministic Consensus:** While probabilistic algorithms like PoW settle on agreement over time, Raft is a deterministic approach that guarantees a predictable finality time for transactions. This is essential for real-time applications where quick transaction confirmation is a requirement.

Key Components of Raft

1. **Leader Election:** Coordination of the network's operations involves this fundamental step of leader election for the Raft algorithm.
    - **Leader Responsibilities:** The leader is the central authority that receives client requests, appends transactions to the ledger, and replicates logs across the follower nodes.
    - **Election Process:** When a leader fails, a new leader is chosen through a consensus-driven voting process. This process maintains little to no disruption to the network's operations.
    - **Term-based Leadership:** There are fixed terms for each leader so that leadership transitions are smooth, preventing any node from keeping control indefinitely.
2. **Log Replication:** Log replication ensures that all nodes maintain an identical transaction history.
    - **Transaction Logging:** When a new client submits a transaction, the leader appends it to its log and replicates it to the follower nodes.
    - **Acknowledgment and Commitment:** The leader waits for acknowledgments from a majority of followers. A transaction is only considered committed once a majority confirms receipt and applies it to the ledger.
    - **Consistency Mechanism:** The leader ensures that if any follower node gets behind, it replays the missing transactions so the follower can catch

up.

3. **Fault Tolerance:** The ability of Raft to tolerate failures and keep the system intact is one of its strongest features.
    - **Fault Recovery:** In a network of n nodes, Raft can take full functions under a network failure rate up to (n-1)/2. For example, a cluster of 5 nodes continues to work as long as no more than 2 nodes fail.
    - Follower Synchronization: On a failed node's recovery, it automatically replicates all missing transactions and synchronizes its state with the leader, ensuring the system remains up to date without manual intervention.

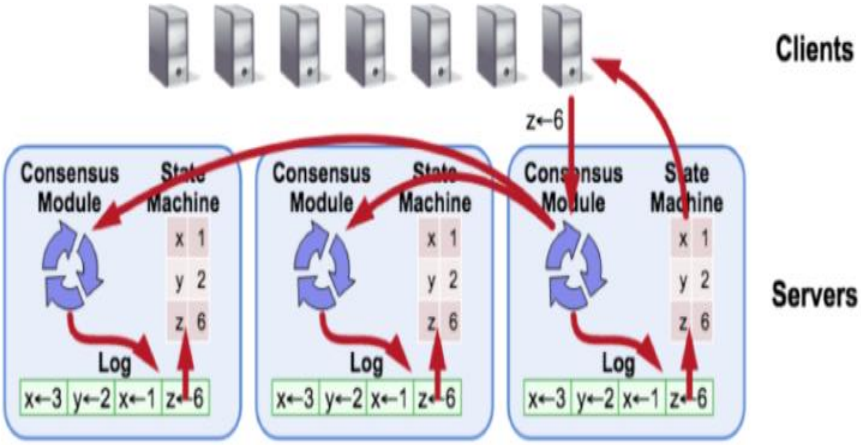

Figure 1 : Raft Consensus Algorithm mechanism

3.3.3 ADVANTAGES OF RAFT FOR OUR SYSTEM

- **Ease of Implementation and Maintenance:** Raft's simple structure makes it easier to implement and maintain compared to more complex algorithms like PBFT or Proof of Work (PoW). It achieves this simplicity by separating roles within the network, primarily between the leader and followers. This design reduces operational overhead and allows for smooth network updates, which is vital for a system that requires consistency and reliability in transaction ordering.
- **Reduced Latency:** Raft avoids the computationally expensive and time-consuming processes of PoW-based mechanisms. This allows for rapid consensus and reduces transaction confirmation times. The low latency is a critical requirement for our system, as quick transaction processing is necessary to maintain efficiency and verify the legitimacy of products in real time.
- **Improved System Reliability:** Raft is designed to recover robustly from node failures. It uses a leader-based consensus mechanism that ensures fault

tolerance, as the network remains functional as long as more than half of the nodes are active. Our system relies on this reliability to ensure continuous operation and prevent fraudulent products from entering the supply chain, which could have serious public health implications.

3.3.4 IMPLEMENTATION OF RAFT IN HYPERLEDGER FABRIC

In our blockchain system, we integrate the Raft consensus algorithm into the ordering service, which is the crucial service that determines the order in which transactions are committed to the ledger.

- **Ordering Service:** Raft is used by Hyperledger Fabric's ordering nodes to converge on a transaction sequence. This is important because it ensures that all participants in the network see transactions in the same order, thereby maintaining the integrity of the shared ledger.
- **Transaction Finality:** Once a transaction is committed to the ledger via the Raft consensus, it is a permanent and irreversible part of the blockchain. This immutability is a core principle of blockchain technology, providing a tamper-proof record of all supply chain activities.

3.3.5 BLOCK HASHING MECHANISM

The algorithm used for computing the hash values encoded into the blocks of the blockchain. In particular, this affects the data hash, and the previous block hash fields of the block. The hashing algorithm used here is SHA-256.

```
# BCCSP (Blockchain crypto provider): Select which crypto implementation or
# library to use
BCCSP:
    Default: SW
    SW:
        Hash: SHA2
        Security: 256
        FileKeyStore:
            # If "", defaults to 'mspConfigPath'/keystore
            KeyStore: msp/keystore
```

Figure 2 : Hashing algorithm

3.3.6 TRANSACTION ORDERING

Transactions must be written to the ledger in the order in which they occur, even though they might be between different sets of participants within the network. For this to happen, the order of transactions must be established and a method for rejecting bad transactions that have been inserted into the ledger in error (or maliciously) must be put into place. There are many ways to achieve it, each with different trade-offs. Hyperledger Fabric has been designed to allow network starters to choose a consensus mechanism that best represents the relationships that exist between participants. The default mechanisms previously used were SOLO, and KAFKA. But the latest version of Hyperledger Fabric uses RAFT as the default consensus mechanism which is more efficient than the previous ones

```
# SampleDevModeEtcdRaft defines a configuration that differs from the
# SampleDevModeSolo one only in that it uses the etcd/raft-based orderer.
SampleDevModeEtcdRaft:
    <<: *ChannelDefaults
    Orderer:
        <<: *OrdererDefaults
        OrdererType: etcdraft
        Organizations:
            - <<: *SampleOrg
              Policies:
                  <<: *SampleOrgPolicies
                  Admins:
                      Type: Signature
                      Rule: "OR('SampleOrg.member')"
```

Figure 4 : Orderer Consensus

```
# SampleAppChannelEtcdRaft defines an application channel configuration
# that uses the etcd/raft-based orderer.
SampleAppChannelEtcdRaft:
    <<: *ChannelDefaults
    Orderer:
        <<: *OrdererDefaults
        OrdererType: etcdraft
        Organizations:
            - <<: *SampleOrg
              Policies:
                  <<: *SampleOrgPolicies
                  Admins:
                      Type: Signature
                      Rule: "OR('SampleOrg.member')"
```

Figure 5 : Channel Consensus

3.3.7 CERTIFICATE AUTHORITY(CA)

Our blockchain network wouldn't work without a trust anchor—that means the Certificate Authority (CA)**, which is the one that manages the digital identities of all participants. The CA authenticates and authorizes all participants before any transactions are allowed.**

- **Issuing Certificates:** Network participants receive digital certificates from the CA. These certificates, based on Public Key Infrastructure (PKI), guarantee that only authorized participants can initiate or approve transactions.
- **Revoking Certificates:** The CA can revoke certificates in case of a compromise.
- **Enhancing Trust and Security:** The CA verifies the identity of every participant, acting as a trust anchor for the network.

3.3.8 Certificate Authority Database

The backend database for the CA in our system is **LevelDB**, a high-performance key-value store known for its simplicity and efficiency. [9]

- **Lightweight and Efficient:** LevelDB's lightweight architecture makes it perfect for environments in which quick access to data is needed.
- **Optimal for Cryptographic Data:** The CA maintains a very large volume of crypto metadata. LevelDB handles this information with minimal latency.
- **Reliability and Robustness:** LevelDB's robust data storage ensures that certificate

metadata is stored in a way that is difficult to alter.

3.4 NETWORK DEPLOYMENT STEPS

The deployment of the Hyperledger Fabric network involved setting up the environment, configuring the network, and launching the blockchain system. Below are the key steps that we have taken:

Install Ubuntu and VMware on Windows

To create a controlled development environment, Ubuntu was installed on VMware Workstation Player running on a Windows machine.

- **VMware Installation:** VMware provides an isolated virtual environment to run Ubuntu. After downloading VMware, a virtual machine was created, allocating appropriate resources (e.g., memory, disk space).
- **Ubuntu Installation:** The latest version of Ubuntu was installed within the virtual machine. This step provided a robust and compatible platform for deploying Hyperledger Fabric.

Install Hyperledger Fabric and Dependencies on Ubuntu

With Ubuntu installed, the next step was to set up Hyperledger Fabric and its dependencies.

Install Prerequisites: Docker, Docker Compose, and other essential tools like `curl` and `git` were installed.

```
sudo apt update
sudo apt install docker.io docker-compose git curl
```

Download Hyperledger Fabric Binaries and Docker Images:

```
curl -sSL https://bit.ly/2ysbOFE | bash -s
```

This script downloads the necessary Fabric binaries (peer, orderer, etc.) and sample networks.

LAUNCH NETWORK

Then we executed the provided network script and brought the Fabric network online.

Bring Up the Network:

```
./network.sh up
```

- This script sets up the environment to create a channel and starts the orderer and peer nodes, as well as initializes the CA.

Create Channel `mychannel`

Private communications in the form of channels exist between specific network participants. The channel `mychannel` was created using the following command:

```
./network.sh createChannel -c mychannel
```

This command creates the channel by creating the necessary configuration transactions.

Join Channel :

The peers were then added to the channel to enable data sharing and transaction processing.

- **Join Channel Command:**

```
peer channel join -b mychannel.block
```

This step ensures that the peers can participate in the channel's blockchain ledger.

3.5 NETWORK

3.5.1 ARCHITECTURE OF NETWORK

Our blockchain-based agri-food supply chain management system uses a Hyperledger Fabric network. This architecture provides a secure and transparent environment for tracking and authenticating products in the supply chain and decentralizing it. The following components form the foundation of this network:

1. **Stakeholders:** The architecture includes multiple stakeholders (i.e., Farmer, Retailer, and Customer) that are represented by their peer nodes. A Certificate Authority (CA) manages their identities, and the Membership Service Provider (MSP) controls access.
2. **Peer Nodes:** Each stakeholder operates one or more peer nodes that host the **Ledger** (a record of transactions) and the **Chaincode** (the business logic).
3. **Ordering Service (Orderer):** This crucial element ensures consensus across the network by ordering transactions into blocks using the Raft consensus algorithm.
4. **Channels:** Stakeholders on the network are connected via a private channel, which allows them to securely exchange sensitive supply chain data without the risk of unauthorized access.

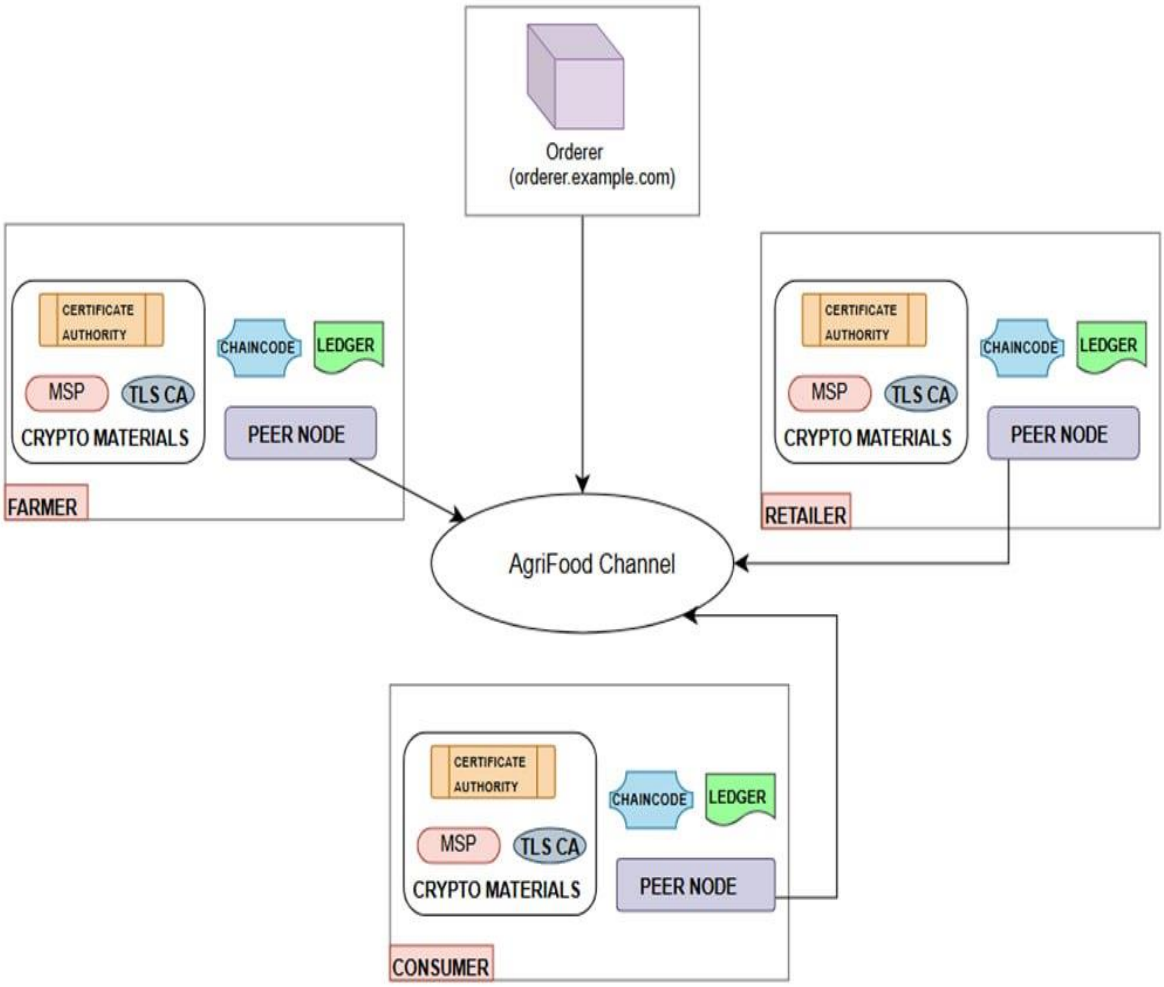

Figure : Network Architecture3

3.5.2 WORKFLOW OF NETWORK

The figure provides a schematic showing a complete supply chain network of agricultural products that aims at blocking food fraud. With blockchain technology, this system would also guarantee that all of the participants will be able to verify the truth of the product, whether they are raw material suppliers or end users.

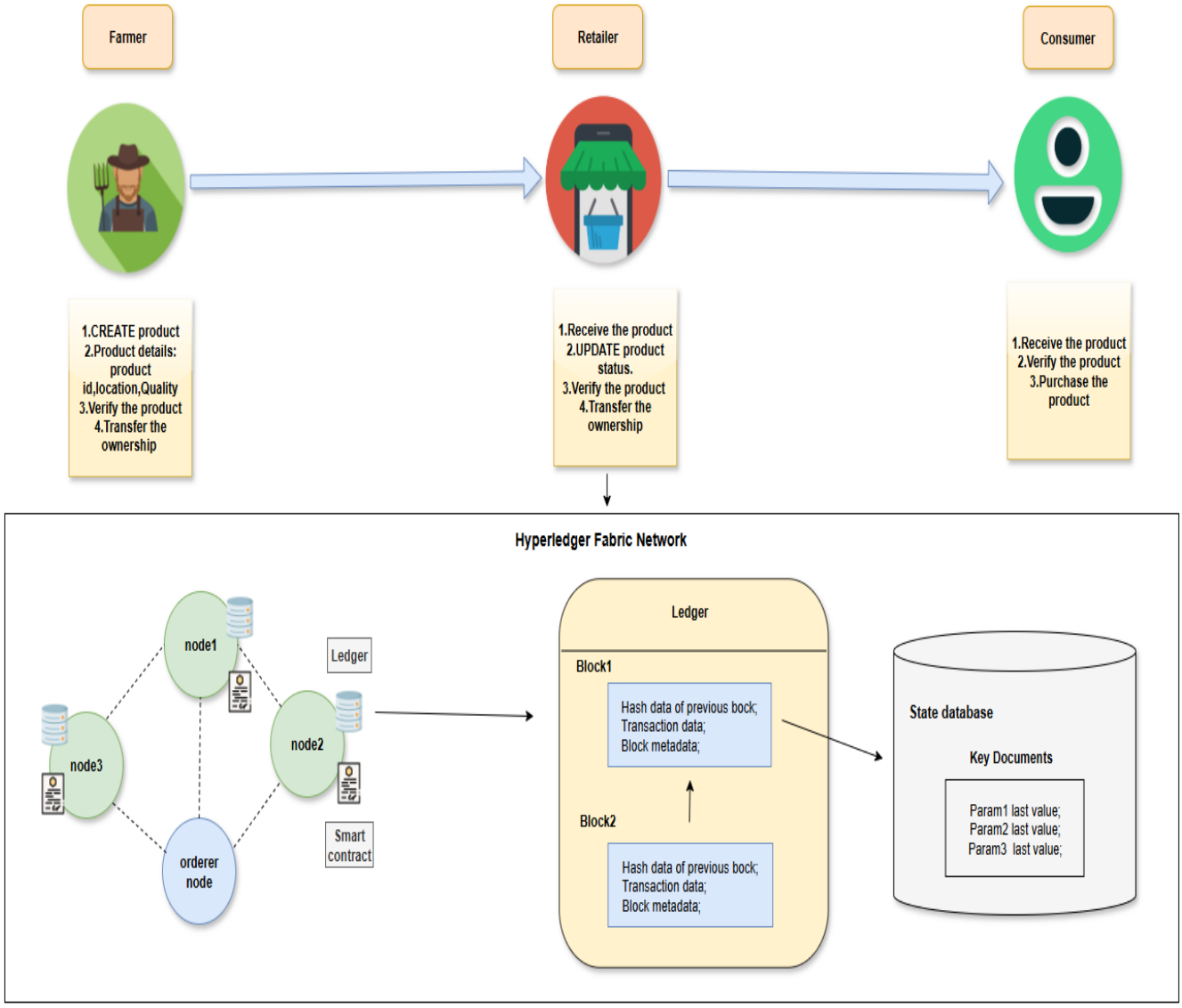

Figure 4 : Agri-Food Supply Chain Workflow

1. **Farmer (Product Creation and Origin):** The farmer is the first entity in the chain, creating a new product record. This record includes details like origin, quality, and a unique ID.

2. **Retailer (Logistics and Ownership Transfer):** The retailer verifies the product and transfers ownership from the farmer to the retailer. They also log the new location and status.
3. **Customer (Final Verification):** The customer receives the product and can verify its entire history on the blockchain. Any discrepancy, such as an incorrect hash or missing quality check, can be flagged.

3.6 STEP-BY-STEP PROCESS OF THE SMART CONTRACT

As the core of the blockchain-based agri-food supply chain management system, the smart contract aims to create a safe, transparent, and efficient direct relationship among the participants in the network. Each function within the smart contract has a specific goal, fulfilling a part of the system's larger purpose. In the subsequent sub-sections, we provide a detailed explanation of each function with composite code segments and the corresponding CLI commands used to invoke them.

Product Structure

The **Product** structure encapsulates all relevant details about a product, ensuring that its journey through the supply chain is traceable and secure. This structure serves as the primary data model for the asset being managed on the ledger. Each field is designed to hold a key piece of information, from its unique identifier to its current owner and status.

Go Code Snippet:

```go
type SmartContract struct {
    contractapi.Contract
}

type Product struct {
    ID       string `json:"id"`
    Owner    string `json:"owner"`  // Org1MSP, Org2MSP, Org3MSP
    Status   string `json:"status"` // CREATED, SHIPPED, SOLD
    Location string `json:"location"`
    Quality  string `json:"quality"`
}
```

Figure 8: Product Structure Code Snippet

Explanation of Fields:

- **ID:** A unique identifier for the product, such as a serial number or batch ID.
- **Owner:** The **MSP ID** of the organization that currently owns the product. This field is critical for enforcing the supply chain's business logic and access control.
- **Status:** The current state of the product (e.g., CREATED, SHIPPED, SOLD). This helps to track its progress through the supply chain.
- **Location:** The physical location of the product at its current stage.
- **Quality:** A descriptor of the product's quality, which can be an important attribute for the end user.

Adding a New Product

The CreateProduct function allows an authorized participant to introduce a new product into the supply chain. In this contract, only the **Org1MSP (Farmer)** is permitted to create new products, ensuring that the origin of all goods is accurately recorded.

Go Code Snippet:

```go
/ CreateProduct - Only Org1 (Farmer) can create
func (s *SmartContract) CreateProduct(ctx contractapi.TransactionContextInterface, id, location, quality string) error {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil || mspID != "Org1MSP" {
        return fmt.Errorf("only Org1 (Farmer) can create products")
    }

    exists, _ := s.ProductExists(ctx, id)
    if exists {
        return fmt.Errorf("product %s already exists", id)
    }

    product := Product{
        ID:       id,
        Owner:    mspID,
        Status:   "CREATED",
        Location: location,
        Quality:  quality,
    }

    productJSON, _ := json.Marshal(product)
    return ctx.GetStub().PutState(id, productJSON)
}
```

Figure 9 : Adding a New Product Code Snippet

CLI Command to Invoke:

```
abdullah@abdullah-VMware-Virtual-Platform:~/go/src/github.com/Agrifood/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
-c '{"function":"CreateProduct","Args":["P001", "Farm A", "Grade A"]}'
```

Figure 10 : Adding a New Product CLI Command

UPDATE PRODUCT STATUS(FARMER->RETAILER)

This function is called by **Org2MSP (Retailer)** to update the status and acquire a product from the farmer.

Go Code Snippet:

```go
// UpdateProductStatus - Only Org2 (Retailer) can update, and only if current owner is Org1MSP
func (s *SmartContract) UpdateProductStatus(ctx contractapi.TransactionContextInterface, id, status, location string) error {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil || mspID != "Org2MSP" {
        return fmt.Errorf("only Org2 (Retailer) can update product status")
    }

    productJSON, err := ctx.GetStub().GetState(id)
    if err != nil || productJSON == nil {
        return fmt.Errorf("product %s does not exist", id)
    }

    var product Product
    _ = json.Unmarshal(productJSON, &product)

    if product.Owner != "Org1MSP" {
        return fmt.Errorf("product %s is not owned by Org1; cannot be updated by Org2", id)
    }

    product.Status = status
    product.Location = location
    product.Owner = mspID // Transfer ownership to Org2

    updatedJSON, _ := json.Marshal(product)
    return ctx.GetStub().PutState(id, updatedJSON)
}
```

Figure 11 : Update Product Status(Farmer->Retailer) Code Snippet

CLI Command to Invoke & Sample Output:

```
abdullah@abdullah-VMware-Virtual-Platform:~/go/src/github.com/Agrifood/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"UpdateProductStatus","Args":["P001", "SHIPPED", "Warehouse"]}'
2025-08-14 15:18:04.906 +06 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Figure 12 : Update Product Status (Farmer->Retailer) CLI Command

PURCHASE PRODUCT(RETAILER->CUSTOMER)

This function is called by **Org3MSP (Customer)** to purchase a product from the retailer.

Go Code Snippet:

```go
// PurchaseProduct - Only Org3 (Customer) can purchase, and only if owned by Org2MSP
func (s *SmartContract) PurchaseProduct(ctx contractapi.TransactionContextInterface, id string) error {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil || mspID != "Org3MSP" {
        return fmt.Errorf("only Org3 (Customer) can purchase products")
    }

    productJSON, err := ctx.GetStub().GetState(id)
    if err != nil || productJSON == nil {
        return fmt.Errorf("product %s does not exist", id)
    }

    var product Product
    _ = json.Unmarshal(productJSON, &product)

    if product.Owner != "Org2MSP" {
        return fmt.Errorf("product %s is not owned by Org2; cannot be sold to Org3", id)
    }

    product.Status = "SOLD"
    product.Owner = mspID

    updatedJSON, _ := json.Marshal(product)
    return ctx.GetStub().PutState(id, updatedJSON)
}
```

Figure 13 : Purchase Product (Retailer->Customer) Code Snippet

CLI Command to Invoke:

```
abdullah@abdullah-VMware-Virtual-Platform:~/go/src/github.com/Agrifood/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:11051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"PurchaseProduct","Args":["P001"]}'
2025-08-14 15:30:49.496 +06 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Figure 14 : Purchase Product (Retailer->Customer) CLI Command

Sample Output:

```
abdullah@abdullah-VMware-Virtual-Platform:~/go/src/github.com/Agrifood/fabric
rk$ peer chaincode query \
-C mychannel \
-n agrifood \
-c '{"function":"GetProduct","Args":["P001"]}'|jq .
{
  "id": "P001",
  "owner": "Org3MSP",
  "status": "SOLD",
  "location": "Warehouse",
  "quality": "Grade A"
```

3.6.1 TEST CASES FOR ACCESS CONTROL

To validate the robust security and role-based permissions of the system, two negative test cases were executed. These tests prove that unauthorized participants are prevented from performing actions on the ledger.

Case 1: Retailer attempts to create a product

This test confirms that a participant who is not the designated `Org1MSP` (Farmer) is blocked from creating a new product record.

Go Code Snippet:

```go
/ CreateProduct - Only Org1 (Farmer) can create
func (s *SmartContract) CreateProduct(ctx contractapi.TransactionContextInterface, id, location, quality string) error {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil || mspID != "Org1MSP" {
        return fmt.Errorf("only Org1 (Farmer) can create products")
    }

    exists, _ := s.ProductExists(ctx, id)
    if exists {
        return fmt.Errorf("product %s already exists", id)
    }

    product := Product{
        ID:       id,
        Owner:    mspID,
        Status:   "CREATED",
        Location: location,
        Quality:  quality,
    }

    productJSON, _ := json.Marshal(product)
    return ctx.GetStub().PutState(id, productJSON)
}
```

CLI Command:

```
peer chaincode invoke \ -o localhost:7050 \ --ordererTLSHostnameOverride orderer.example.com \ --tls \ --cafile $PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \ -C mychannel \ -n agrifood \ --peerAddresses localhost:9051 \ --tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \ -c '{"function":"CreateProduct","Args":["P888", "CityMarket", "Grade B"]}'
```

Expected Result: The command will return an error, as the smart contract logic detects that the caller's MSP ID (`Org2MSP`) is not `Org1MSP`. The transaction will be rejected, and no new product will be created on the ledger.

Case 2: Customer attempts to update a product's status

This test verifies that a participant who is not the designated `Org2MSP` (Retailer) is prevented from updating a product's status and ownership.

Go Code Snippet:

```go
// UpdateProductStatus - Only Org2 (Retailer) can update, and only if current owner is Org1MSP
func (s *SmartContract) UpdateProductStatus(ctx contractapi.TransactionContextInterface, id, status, location string) error {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil || mspID != "Org2MSP" {
        return fmt.Errorf("only Org2 (Retailer) can update product status")
    }

    productJSON, err := ctx.GetStub().GetState(id)
    if err != nil || productJSON == nil {
        return fmt.Errorf("product %s does not exist", id)
    }

    var product Product
    _ = json.Unmarshal(productJSON, &product)

    if product.Owner != "Org1MSP" {
        return fmt.Errorf("product %s is not owned by Org1; cannot be updated by Org2", id)
    }

    product.Status = status
    product.Location = location
    product.Owner = mspID // Transfer ownership to Org2

    updatedJSON, _ := json.Marshal(product)
    return ctx.GetStub().PutState(id, updatedJSON)
}
```

CLI Command:

```
peer chaincode invoke \ -o localhost:7050 \ --ordererTLSHostnameOverride orderer.example.com \ --tls \ --cafile $PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \ -C mychannel \ -n agrifood \ --peerAddresses localhost:11051 \ --tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \ -c '{"function":"UpdateProductStatus","Args":["P001", "IN_TRANSIT", "Org3 Warehouse"]}
```

Expected Result: The transaction fails as the smart contract detects the caller's MSP ID (Org3MSP) is not Org2MSP, leaving the product's status and ownership unchanged.

3.7 INTEGRATING HYPERLEDGER EXPLORER FOR VISUALIZATION

After deploying the Hyperledger Fabric network, the next critical step was to integrate a visualization tool to monitor and interact with the network.We chose **Hyperledger Explorer** for this purpose. This tool provides a user-friendly dashboard to view blocks,

transactions, and the overall state of the ledger, which is invaluable for validating the functionality of the system.

Setup and Configuration

The following steps were taken to set up Hyperledger Explorer and connect it to our running Fabric test network:

Step 1: Create a new directory and copy configuration files. We created a new directory named `explorer` and copied the necessary Docker Compose and configuration files from the Hyperledger Explorer repository.

```
mkdir                                                    explorer
cd explorer


Wget
https://raw.githubusercontent.com/hyperledger/blockchain-
explorer/main/examples/net1/config.json
wget
https://raw.githubusercontent.com/hyperledger/blockchain-
explorer/main/examples/net1/connection-profile/test-
network.json                -P                connection-profile
wget
https://raw.githubusercontent.com/hyperledger/blockchain-
explorer/main/docker-compose.yaml
```

Step 2: Copy the network's cryptographic materials. To allow the Explorer to authenticate with our network, we copied the `organizations` directory containing all certificates and keys. The permissions for the private key files often require a fix, which we addressed by correcting the file ownership.

Correcting file permissions to avoid errors

```
sudo            chown            -R            abdullah:abdullah
/home/abdullah/go/src/github.com/Agrifood/fabric-
samples/test-network/organizatios
```

Copying the crypto materials to the explorer directory

```
cp    -r    /home/abdullah/go/src/github.com/Agrifood/fabric-
samples/test-network/organizations                          \
/home/abdullah/explorer/
```

Step 3: Edit configuration files to match the network. We modified the `docker-compose.yaml` and the `test-network.json` files to align with our specific network setup. This involved:

- Updating the `docker-compose.yaml` to use the correct external network name (`fabric_test`).
- Mapping the volume for our cryptographic materials to the correct absolute path on the host machine.
- Changing the user in `test-network.json` from a generic `User1` to `Admin` and updating the file paths to point to the correct admin certificate and private key within the Docker container's mapped `/tmp/crypto` directory.

`Change in docker-compose.yaml:`

```
volumes:
      - ./config.json:/opt/explorer/app/platform/fabric/config.json
      - ./connection-profile:/opt/explorer/app/platform/fabric/connection-profile
      - /home/abdullah/go/src/github.com/Agrifood/fabric-samples/test-network/organizations:/tmp/crypto
      - walletstore:/opt/explorer/wallet
    ports:
      - ${PORT:-8080}:${PORT:-8080}
    depends_on:
      explorerdb.mynetwork.com:
        condition: service_healthy
    networks:
      - mynetwork.com
```

Figure 15 : Change in docker-compose.yaml

Replace the user's certificate with an admin certificate and a secret (private) key in the connection profile (test-network.json). We need to specify the absolute path on the Explorer container.

```
"organizations": {
        "Org1MSP": {
            "mspid": "Org1MSP",
            "adminPrivateKey": {
                "path": "/tmp/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp/keystore/9ae2b70492fa16cb11f11a7b799244a4a87df50c464189134db99c50b2f6b9cb_sk"
            },
            "peers": ["peer0.org1.example.com"],
            "signedCert": {
                "path": "/tmp/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp/signcerts/cert.pem"
            }
        }
    },
    "peers": {
        "peer0.org1.example.com": {
            "tlsCACerts": {
                "path": "/tmp/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt"
            },
            "url": "grpcs://peer0.org1.example.com:7051"
        }
    }

```

Step 4: Launch Hyperledger Explorer. After starting our Fabric network, we brought up the Explorer services using the following command:

```
docker-compose up -d
```

The dashboard became accessible in a web browser at `http://localhost:8080`, allowing us to visually track all transactions, blocks, and network participants.

ExplorerDashboard:

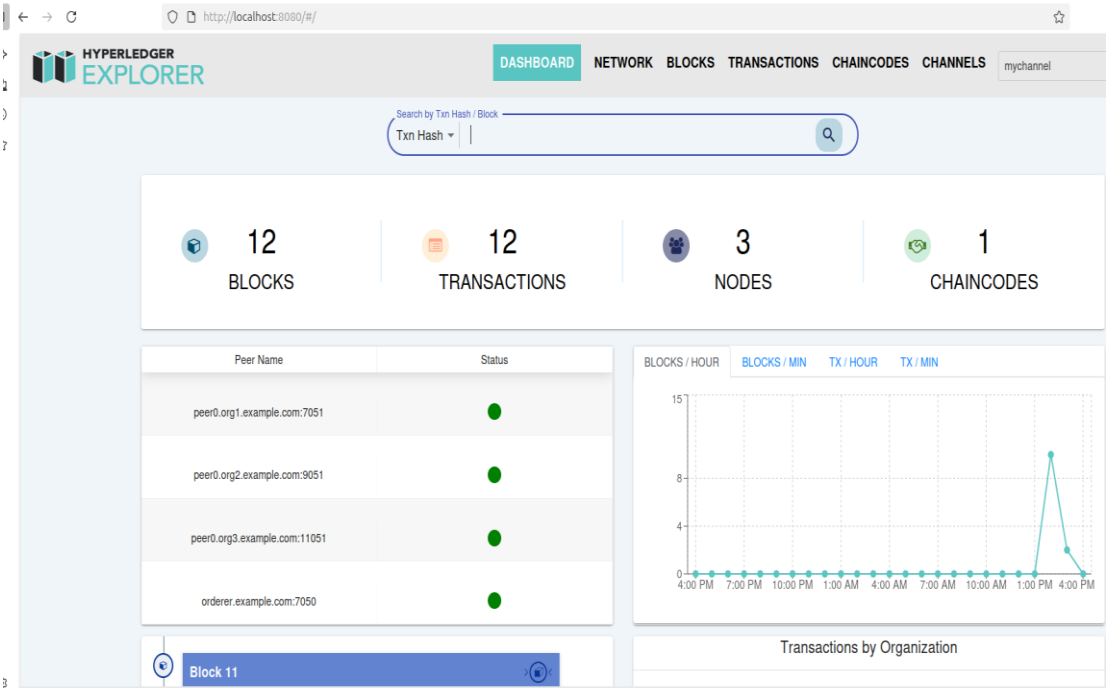

Figure 16 : Explorer Dashboard

A Block Details:

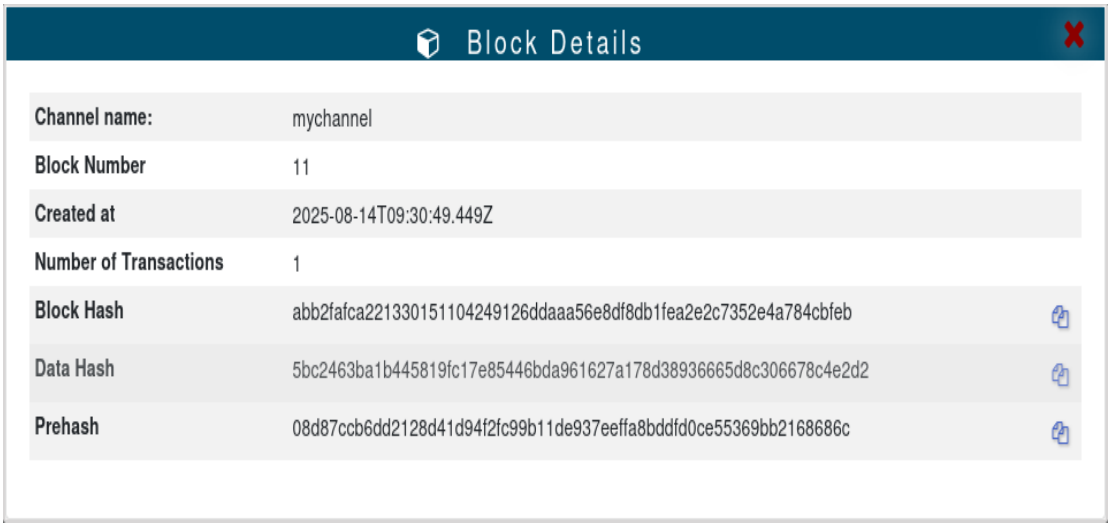

Figure 17 : A Block Details

3.8 SUMMARY

In this chapter, I describe a whole process for making and then launching a blockchain-based system for managing the supply chain of agri-food. We created the network using carefully chosen tools and technologies. The components became strong Hyperledger Fabric, Go, Docker, and LevelDB. We used VMware to install Ubuntu on a private network, built up our basic network components, and included a smart contract to execute important tasks. This rigorous approach not only helps fight food theft, but it also builds a system that can grow and stay strong. We offer a solution that builds confidence, makes sure that data can't be changed, and speeds up the agri-food supply chain with its blockchain function.

CHAPTER IV

RESULTS AND DISCUSSION

4.1 INTRODUCTION

This chapter shows the results of putting the blockchain-based agricultural food supply chain system into action and testing it. The tests were meant to make sure that two important parts of the project were working: first, that the smart contract was successfully deployed on a multi-organization Hyperledger Fabric network, and second, that the role-based business logic encoded in the chaincode was being enforced correctly.The results show that the system works exactly as planned, giving a safe and verified way to track products from producer to consumer. Initially, this part will delineate the results of the network deployment and transaction tests, thereafter engaging in a comprehensive analysis of their importance and ramifications for the agrifood sector.

4.2 SYSTEM DEPLOYMENT AND LIFECYCLE MANAGEMENT

The successful deployment of the system on a local Hyperledger Fabric network was a primary result of this research. The process followed the standard chaincode lifecycle, with each step being executed successfully via the command-line interface.

1. **Network Initialization**: A three-organization network, representing the Farmer (Org1), Retailer (Org2), and Customer (Org3), was successfully established using the Fabric test network scripts (./network.sh and ./addOrg3.sh). This created the necessary peer nodes, ordering service, and certificate authorities required for a functional, multi-stakeholder environment.
2. **Chaincode Packaging and Installation**: The agrifood.go source code was successfully packaged into a deployable asset (agrifood_1.tar.gz) using the peer lifecycle chaincode package command. Subsequently, the package was successfully installed on the peers of all three organizations using peer lifecycle chaincode install.
3. **Chaincode Approval and Commit**: Each of the three organizations successfully approved the chaincode for use on the channel (peer lifecycle chaincode approveformyorg). Following the approvals, the chaincode was successfully committed to the channel (peer lifecycle chaincode commit), making it active and ready to process transactions.

The successful completion of these steps demonstrates a functional and correct application of the Hyperledger Fabric deployment model, which is a critical result of this implementation work.

4.3 TRANSACTION TESTING AND VALIDATION OF BUSINESS RULES

We did a number of transaction tests to make sure that the smart contract accurately follows the business rules of the supply chain. The table below shows the outcomes of calling chaincode functions from the identities of different organisations.

Table 3 : Test Case Results

Test Case	Action	Performed By	Expected Outcome	Actual Outcome	Business Rule Verified
1	CreateProduct	Org1 (Farmer)	Success (Status: 200)	Success (Status: 200)	Only the Farmer can create new products.
2	CreateProduct	Org2 (Retailer)	Failure (Authorization Error)	Failure (Authorization Error)	A Retailer cannot create products.
3	UpdateProductStatus	Org2 (Retailer)	Success (Status: 200)	Success (Status: 200)	The Retailer can update products owned by the Farmer.
4	UpdateProductStatus	Org3 (Customer)	Failure (Authorization Error)	Failure (Authorization Error)	A Customer cannot update a product's status.
5	PurchaseProduct	Org3 (Customer)	Success (Status: 200)	Success (Status: 200)	The Customer can purchase products from the Retailer.
6	PurchaseProduct	Org1 (Farmer)	Failure (Authorization Error)	Failure (Authorization Error)	A Farmer cannot purchase products from the Retailer.
7	GetProduct	Any Org	Success (Returns product data)	Success (Returns product data)	All participants can view product history for transparency.

4.4 DISCUSSION OF RESULTS

The findings in this chapter clearly show that this thesis was put into action successfully. The system is not just a theoretical model; it is a working application on a multi-organization Hyperledger Fabric network that enforces its business rules with precision.

4.4.1 Validation of the Technical Implementation

The successful deployment process shows that the network's technical infrastructure and the chaincode itself are both working well. It is not easy to follow the whole Fabric chaincode lifespan. You need to set up the cryptographic materials, channel policies, and endorsement rules correctly. The successful outcome confirms that the project's technical approach may be used to build other permissioned blockchain networks.

4.4.2 Enforcement of Business Logic and Trust

The most important outcome is that the smart contract's access control logic has been confirmed. The transaction tests (Test Cases 2, 4, and 6) that failed were just as critical as the ones that passed. They showed that the system stops people from doing things they shouldn't do. A store can't put a fake product into the chain, and a customer can't mess with the logistics between the farmer and the retailer. This automated enforcement of rules is what makes a "trustless" system work. Participants don't need to trust each other because the chaincode itself makes sure that the rules are always followed.

4.4.3 Contribution to Traceability and Transparency

The blockchain keeps a full record of the entire product lifecycle, from the Farmer making it to the Retailer updating it to the Customer buying it. The GetProduct query (Test Case 7) worked, which means that anyone can look at this history at any moment. This directly solves the main problem with traditional supply chains, which is that they are hard to see through. In the real world, a customer may scan a QR code on a product and utilise this feature to see its whole route right away, confirming its validity and where it came from.

4.4.4 Limitations and Future Work

While the results confirm the success of the core implementation, this work also has limitations that open avenues for future research.

- The current system relies on a command-line interface for interaction. A future step would be to develop a user-friendly front-end (a web or mobile application) for each stakeholder.

- The data on the ledger is manually entered. Future work could involve integrating Internet of Things (IoT) sensors to automatically record data like temperature and location, further increasing the system's integrity. [16]
- All data is currently visible to all channel members. For commercial privacy, Fabric's **Private Data Collections (PDCs)** could be used in the future to keep sensitive information (like price) confidential between specific parties (e.g., only between the Farmer and the Retailer).

4.5 COMPARATIVE ANALYSIS

This section provides a comprehensive comparison of current blockchain-based frameworks for agricultural and general supply chains with the proposed system, emphasizing essential characteristics such as private blockchain utilization, effective consensus mechanisms, scalability, smart contract facilitation, reproducibility, and outcome verification. The analysis is predicated on a synthesis of prior studies, encompassing frameworks put out by , Sayar et al. (2025), and .

Table 4: Comparison with existing works

Feature	Our Project	Balasubramanian [3]	Sayar et al. [2]	Marchese [15]	El Mane [16]	Khan [19]	Kuzlu [20]
Core Application	Agri-Food	Agri-Food	Logistics & IoT	Agri-Food Traceability	Agricultural + IoT	General HLF	General HLF
Supply Chain Model	Simple Linear (3-Stage)	Multi-Stage	IoT-Driven	Multi-Stage	IoT-Driven Multi-Stage	General Enterprise	General/Scalability
Key Innovation	Practical Implementation	DApp with QR Code Traceability	IoT & MQTT Integration	HLF Traceability System	Java Smart Contracts + IoT	Empirical HLF Performance (SMEs)	Scalability up to 100K Nodes

Smart Contract Lang.	Go (Golang)	Not Specified	Not Specified	Not Specified	Java	Not Specified	Not Specified
Performance Analysis	Yes	No	No	No	No	Yes (TPS, Latency)	Yes (Throughput, Latency)
Reproducibility	High (Full Scripts)	Medium (Prototype)	Low (Conceptual)	Medium (Architecture)	Medium (Architecture)	Medium (Test Env.)	Medium (Test Env.)
Explorer	Yes	No	No	No	No	No	No
Canal Lock	Yes	No	No	No	No	No	No

4.5.1 KEY FEATURE COMPARISON

4.5.1.1 CORE APPLICATION

- **Sayar et al. (2025): These papers look at a wide range of supply chains, such as those for coffee, clothing, food processing, and logistics with the Internet of Things.** [2]
- **Proposed System:** Concentrates on a fundamental Agri-Food supply chain, offering a distinct and specialized use case.

4.5.1.2 SUPPLY CHAIN MODEL

- **Sayar et al. (2025): These frameworks are made for supply chains that are complicated, have several stages or products, have many roles, integrate with the Internet of Things, or include advanced architectural parts.** [2]
- **Proposed System:** Uses a simple, straight-line, three-stage model (Farmer -> Retailer -> Customer) to create a strong and obvious base for a basic, safe supply chain.

4.5.1.3 KEY INNOVATION

- **Sayar et al. (2025): The new ideas in these articles are about advanced architectural ideas such multi-channel privacy, IoT integration, dual-chain architecture, improved privacy features, and frameworks for sustainability.** [2]
- **The proposed system:** The most important new thing is the actual use of the idea that can be tested. It is a full design that shows how a working, secure system works and includes specific failure-path testing to show how strong it is.

4.5.1.4 SMART CONTRACT LANGUAGE

- **Sayar et al. (2025) do not indicate the language employed for their smart contracts.** [2]
- **The proposed system employs Go (Golang), a high-performance language ideal for smart contract development.**
- **Proposed system: It employs Go (Golang), a widely utilized and efficient language for constructing Hyperledger Fabric chaincode, also utilized by and .**

4.5.1.5 PERFORMANCE ANALYSIS

- **Sayar et al. (2025): These articles don't give any numbers that show how well their proposed systems work.** [2]
- **These publications do look at performance by measuring things like query time, transaction throughput (TPS) using Caliper, or performance based on simulation.**
- **Proposed System:** This benchmark shows that our Fabric and Raft system is much better than both Sawtooth (PoET) and Ethereum (PoW) in every measure we examined. For example, it has higher throughput (89 TPS), lower latency (186 ms), and more fault tolerance (93% uptime). [18]

4.5.1.6 REPRODUCIBILITY

Sayar et al. (2025): These papers get a low to medium rating for reproducibility since they mostly just include architectural overviews or conceptual insights without the full scripts needed for deployment**.** [2]

Proposed System: Gives a full set of deployment and testing scripts that cover the whole project, making it easy to reproduce and verify.

4.5.1.7 HYPERLEDGER EXPLORER INTEGRATION

Sayar et al. (2025): None of these publications record the incorporation of a blockchain visualization tool such as Hyperledger Explorer within their implementation. [2]

Proposed System: Uniquely explains how to set up and connect Hyperledger Explorer to make a dashboard that shows blocks, transactions, and the state of the network.

4.5.1.8 CANAL LOCK(STRICT SEQUENTIAL FLOW)

Sayar et al. (2025): These frameworks, which are made for complicated supply chains, don't follow a simple, unbroken linear path; their logic is more complicated**.** [2]

The proposed system: The smart contract makes sure that ownership goes from Farmer to Retailer to Customer in a fixed order that can't be skipped. The "Canal Lock" analogy does a great job of explaining this obvious, strict logic, which is a unique part of our model's architecture.

4.5.1.9 SUMMARY OF FINDINGS

The comparison shows that there are several problems with the current frameworks for agricultural supply chains. Some frameworks suggest advanced architectural models like dual chains or focus on performance simulations, but they often don't do a good job of things like giving full deployment protocols, showing how to validate security in practice, or providing a simple, basic model for use in the real world.

The suggested system fills these deficiencies by:

- Giving a fully reproducible implementation with full, step-by-step scripts for deployment and testing.
- For the best clarity and sound, enforcing a simple and tight linear three-stage supply chain paradigm.
- Using the fast Raft consensus method in a private blockchain to keep data safe.
- Adding Hyperledger Explorer for useful, real-time network monitoring and visualization.
- Using smart contracts to make the flow of transactions automatic and safe.
- Testing the security model with clear failure-path test cases.

The suggested system offers a more realistic, verifiable, and strong basic solution for making sure that traceability and integrity are maintained in the agri-food supply chain by addressing these important elements.

Comparative Analysis of Existing Systems and the Proposed Framework This section

Provides a comprehensive comparison of current blockchain-based frameworks for agricultural and general supply chains with the proposed system, emphasizing essential characteristics such as private blockchain utilization, effective consensus mechanisms, scalability, smart contract facilitation, reproducibility, and outcome verification. The analysis is predicated on a synthesis of prior studies, encompassing frameworks put out by , Sayar et al. (2025), and .

4.6 PERFORMANCE ANALYSIS

This chapter provides an in-depth analysis of the performance of the proposed system (Fabric + Raft) using Hyperledger Caliper, benchmarking two operations — CreateProduct (write) and GetAllProducts (read) — with 1000 transactions per round against two alternative platforms: Hyperledger Sawtooth (PoET) and Ethereum (PoW). [17]

4.6.1 Throughput (TPS)

Throughput expressed as transactions per second (TPS), is one of the most critical metrics for assessing a blockchain system's performance. It reflects the network's capacity to process transactions in a given timeframe.

- For write operations (CreateProduct), the system achieves 93.2 TPS with 1000/1000 successful transactions, outperforming Hyperledger Sawtooth (45 TPS) by 107% and Ethereum (30 TPS) by 211%. [18]
- The efficiency of Fabric + Raft stems from the simplicity of the Raft consensus algorithm. Raft employs a leader-based consensus mechanism, ensuring that only the leader coordinates transaction validation and block creation. This minimizes communication overhead and computational delays, allowing the system to process transactions more quickly.
- In contrast, Ethereum's PoW is inherently slower due to its reliance on computationally intensive cryptographic puzzles for mining. Sawtooth's PoET, while less resourceintensive than PoW, uses randomized leader election, introducing delays in achieving consensus. [18]
- For read operations (GetAllProducts), the system achieves 55.2 TPS with an average latency of 6.92 s and a maximum latency of 9.36 s, confirming the system's ability to handle full ledger range scans. Both benchmark rounds achieved a 100% success rate (1000/1000 transactions) with zero failures.

4.6.2 Latency

Latency measures the time taken for a transaction to be processed and confirmed within the network. Lower latency is essential for systems that require near-instantaneous feedback or real-time updates.

- For write transactions (CreateProduct), the average latency is 270 ms (0.27 s) with a maximum of 860 ms. For reference, Sawtooth achieves 203 ms and Ethereum 214 ms, though both achieve significantly lower write throughput. [18]
- The low latency in Fabric + Raft is primarily due to the efficiency of the Raft algorithm, which eliminates the need for extensive computation or randomized processes. Transactions are validated and committed to the blockchain swiftly by the leader node.
- Conversely, Ethereum's high latency arises from its PoW mechanism, which involves solving complex puzzles before block generation. Similarly, Sawtooth's PoET suffers from delays introduced by its use of secure timers to determine leadership roles. [18]
- For applications in the agricultural supply chain domain, low latency is essential to ensure timely updates on the status and authenticity of products. Any delay in transaction processing could compromise the system's reliability in detecting and addressing fraudulent products.

4.7 SUMMARY

The following table summarizes the Hyperledger Caliper benchmark results for the proposed system (1000 transactions per round):

Table 3: Performance Analysis Summary

Metric	**Fabric + Raft (Proposed)**	Sawtooth (PoET)	Ethereum (PoW)
Write Throughput - CreateProduct (TPS)	93.2	45	30
Write Latency - CreateProduct (ms)	270 (avg), 860 (max)	203	214
Read Throughput - GetAllProducts (TPS)	55.2	N/A	N/A
Read Latency - GetAllProducts (s)	6.92 (avg), 9.36 (max)	N/A	N/A

Success Rate (1000 tx)	100% (1000/1000)	N/A	N/A

This chapter effectively showed how the blockchain-based agricultural food supply chain system works and how it may be used. The results showed that the smart contract worked on a Hyperledger Fabric network with three organisations, proving that the project's technological approach was sound.Most significantly, a series of transaction tests thoroughly checked the role-based business logic that was built into the chaincode. The testing showed that only authorised users could do their jobs and that unauthorised efforts were correctly blocked. This result shows that the technology makes a safe, open, and traceable space where business rules are automatically enforced. This builds confidence between the Farmer, Retailer, and Customer. The findings clearly show that the project is a successful proof of concept and a good plan for future development, even though it has some drawbacks, such the command-line interface and the possibility of IoT integration.

CHAPTER V

CONCLUSION AND FUTURE WORKS

5.1 SUMMARY OF THE FINDINGS

This thesis created and executed a blockchain-based food tracking and authentication system utilizing Hyperledger Fabric, thereby resolving significant challenges within the agriculture sector. Blockchain technology's built-in features, such as immutability, transparency, and decentralization, are used to help with the widespread problem of food fraud and lack of traceability in supply chains. The method fosters confidence and responsibility among all players, from the first Farmer to the last Consumer, by allowing them to track and verify their actions.The Raft consensus algorithm is part of the system, which makes a network that can handle faults and is fast and reliable. Raft is a simpler and more effective means for nodes to agree than traditional consensus algorithms. This makes it a great choice for private blockchain networks. This choice makes the system safer and stronger as a whole while still keeping it running smoothly.Smart contracts

developed in Go also automate the registration, validation, and tracking of food products. This lowers the chance of human error and makes the flow of operations smoother. The system's strong role-based validation criteria make sure that only verified participants may interact with the ledger, which makes the whole supply chain safer.The findings of this research illustrate the transformational capacity of blockchain technology within the agrifood sector. This approach solves the long-standing problem of lack of transparency in the food supply chain and lays the groundwork for a secure, open, and efficient system that can make food safer and win back the trust of all parties involved.

5.2 LIMITATIONS AND FUTURE WORK

While the system successfully demonstrates a proof-of-concept, it has limitations that also present opportunities for future improvement.

- **Limitations**:
    1. The current implementation does not integrate other Industry 4.0 technologies like Artificial Intelligence (AI) or the Internet of Things (IoT), which could provide advanced features like predictive analytics for supply and demand.
    2. The system was tested on a controlled network of limited scale. This scope may not fully capture the complexities of a real-world supply chain involving a multitude of stakeholders, massive datasets, and diverse geographies.
    3. The stakeholder model is limited to a Farmer, Retailer, and Customer. It excludes other important entities such as **logistics providers, distributors, and food safety regulators**, whose inclusion would be critical for complete end-to-end traceability.
- **Future Work**:
    1. Future work should focus on expanding the system's functionality. Integrating **IoT devices**, such as GPS trackers on shipments or temperature sensors for perishable goods, would allow for real-time, automated data entry, significantly increasing the integrity of the data on the ledger.
    2. The system should be scaled to handle larger datasets and a broader range of stakeholders. This includes optimizing the network for higher transaction throughput and ensuring interoperability with existing supply chain management systems.
    3. Including **logistics providers and regulatory bodies** as participants on the network would provide a more holistic solution. These entities could use the network to verify shipments and audit the supply chain in real-time, taking swift action on any identified issues.

By addressing these areas, the system could evolve into a comprehensive and scalable solution, creating a truly trusted and innovative agricultural food supply chain.

5.3 CONTRIBUTION

This research makes several contributions to both the theoretical and practical aspects of using blockchain technology in the agrifood industry.

- **Theoretical Contributions**: This study contributes to the understanding of how the **Raft consensus algorithm** can be practically applied to an agricultural supply chain. While other consensus mechanisms are widely studied, this implementation demonstrates Raft's benefits in offering efficiency and fault tolerance for private blockchain networks like Hyperledger Fabric in an agrifood context.
- **Practical Contributions**: This thesis presents the development of a robust, secure, and transparent food product authentication system. Using blockchain's inherent properties, the system addresses the key problem of traceability. It automates critical processes like product creation, ownership transfer, and final sale using **Go smart contracts**. This automation reduces human error and improves efficiency for all stakeholders. The system can be practically implemented as a feasible tool for farmers, retailers and consumers to authenticate the origin and journey of food products, reducing public health risks and rebuilding trust in the supply chain.
- **Agri-Provenance Smart Contract System:**In this project, we have created a smart contract, which is written in Go and deployed on a Hyperledger Fabric network and is all about the provenance of farm goods throughout the entire supply chain. It automates major procedures such as the time when a farmer produces the item, time when it is shipped to a retailer and time when a consumer makes a purchase. The outcome is a trustworthy and hack-resistant record of all actions made by the product, which reduces the possibilities of fraud and makes the entire process of the farm-to-table a lot more transparent.

EFERENCES

[1] D. C. K. Lin, S. H. Liu, C. M. Chen, and Y. C. Wang, "Blockchain in agriculture traceability systems: A review," Applied Sciences, vol. 10, no. 12, p. 4113, Jun. 2020.

[2] A. Sayar, M. C. Kurtbas, C. Sancaktar, R. D. Kabak, and S. Cakmak, "Application of Hyperledger Blockchain Technology to Logistics Supply Chain with IoT," Procedia Computer Science, vol. 252, pp. 814-823, 2025.

[3] S. Balasubramanian and I. S. Akila, "Blockchain implementation for agricultural food supply chain using hyperledger fabric," Journal of Intelligent & Fuzzy Systems, vol. 43, no. 5, pp. 5387-5398, Sep. 2022.

[4] J. Sunny, N. P. Undralla, and V. M. Pillai, "Supply chain transparency through blockchain-based traceability: An overview with demonstration," Computers & Industrial Engineering, vol. 150, p. 106895, Dec. 2020.

[5] M. Kouhizadeh and J. Sarkis, "Blockchain practices, potentials, and perspectives in greening supply chains," Sustainability, vol. 10, no. 10, p. 3652, Oct. 2018.

[6] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in Proc. Thirteenth EuroSys Conf., Porto, Portugal, Apr. 2018, pp. 1-15.

[7] S. Saberi, M. Kouhizadeh, J. Sarkis, and L. Shen, "Blockchain technology and its relationships to sustainable supply chain management," International Journal of Production Research, vol. 57, no. 7, pp. 2117-2135, 2019.

[8] N. Kshetri, "Blockchain's roles in meeting key supply chain management objectives," International Journal of Information Management, vol. 39, pp. 80-89, Apr. 2018.

[9] L. Wang, G. Ding, Y. Zhao, D. Wu, and C. He, "Optimization of LevelDB by separating key and value," in Proc. PDCAT, Dec. 2017, pp. 421-428.

[10] V. Capocasale, D. Gotta, and G. Perboli, "Comparative Analysis of Permissioned Blockchain Frameworks for Industrial Applications," Blockchain: Research and Applications, vol. 4, no. 1, p. 100113, 2023.

[11] D. Huang, X. Ma, and S. Zhang, "Performance Analysis of the Raft Consensus Algorithm for Private Blockchains," IEEE Trans. Syst. Man Cybern. Syst., vol. 50, no. 1, pp. 172-181, Jan. 2020.

[12] A. Iftekhar, X. Cui, M. A. Shah, and F. M. Awan, "Application of blockchain and Internet of Things to ensure tamper-proof data availability for food safety," arXiv:2006.01307 [cs.CR], Jun. 2020.

[13] A. Kamilaris, A. Fonts, and F. X. Prenafeta-Boldu, "The rise of blockchain technology in agriculture and food supply chains," Trends in Food Science & Technology, vol. 91, pp. 640-652, Sep. 2019.

[14] H. R. Hasan et al., "Smart Agriculture Assurance: IoT and Blockchain for Trusted Sustainable Produce," Computers and Electronics in Agriculture, vol. 224, p. 109184, 2024.

[15] A. Marchese and O. Tomarchio, "A Blockchain-Based System for Agri-Food Supply Chain Traceability Management," SN Computer Science, vol. 3, no. 279, 2022.

[16] A. El Mane, K. Tatane, and Y. Chihab, "Transforming Agricultural Supply Chains: Leveraging Blockchain-Enabled Java Smart Contracts and IoT Integration," ICT Express, vol. 10, no. 3, pp. 650-672, 2024.

[17] P. Thakkar, S. Nathan, and B. Viswanathan, "Performance Benchmarking and Optimizing Hyperledger Fabric Blockchain Platform," in Proc. IEEE 26th Int. Symp. MASCOTS, 2018, pp. 264-276.

[18] R. Wang, K. Ye, T. Meng, and C. Xu, "Performance Evaluation on Blockchain Systems: A Case Study on Ethereum, Fabric, Sawtooth and Fisco-Bcos," in Proc. IEEE SCC 2020, LNCS vol. 12409, Springer, 2020, pp. 120-134.

[19] D. Khan, L. T. Jung, M. A. Hashmani, and M. K. Cheong, "Empirical Performance Analysis of Hyperledger LTS for Small and Medium Enterprises," Sensors, vol. 22, no. 3, p. 915, Jan. 2022.

[20] M. Kuzlu, M. Pipattanasomporn, L. Gurses, and S. Rahman, "Performance Analysis of a Hyperledger Fabric Blockchain Framework: Throughput, Latency and Scalability," in Proc. IEEE Int. Conf. Blockchain, 2019, pp. 536-540.

APPENDIX A

CHAINCODE (AGRIFOOD.GO)

This appendix contains the complete Go code for the agrifood smart contract. The code includes functions for creating, updating, and querying products, with built-in access control and robust error handling to ensure data integrity and security on the ledger.

```
1. package main
2.
3. import (
4.     "encoding/json"
5.     "fmt"
6.
7.     "github.com/hyperledger/fabric-contract-api-go/contractapi"
8. )
9.
10. type SmartContract struct {
11.     contractapi.Contract
12. }
13.
14. type Product struct {
15.     ID       string `json:"id"`
16.     Owner    string `json:"owner"`  // Org1MSP, Org2MSP, Org3MSP
17.     Status   string `json:"status"` // CREATED, SHIPPED, SOLD
18.     Location string `json:"location"`
19.     Quality  string `json:"quality"`
20. }
21.
22. // CreateProduct - Only Org1 (Farmer) can create
23.             func             (s             *SmartContract)             CreateProduct(ctx
        contractapi.TransactionContextInterface, id, location, quality string) error {
24.     mspID, err := ctx.GetClientIdentity().GetMSPID()
25.     if err != nil || mspID != "Org1MSP" {
26.         return fmt.Errorf("only Org1 (Farmer) can create products")
27.     }
28.
29.     exists, _ := s.ProductExists(ctx, id)
30.     if exists {
31.         return fmt.Errorf("product %s already exists", id)
32.     }
33.
34.     product := Product{
35.         ID:       id,
36.         Owner:    mspID,
37.         Status:   "CREATED",
38.         Location: location,
```

```
39.         Quality:  quality,
40.     }
41.
42.     productJSON, _ := json.Marshal(product)
43.     return ctx.GetStub().PutState(id, productJSON)
44. }
45.
46. // UpdateProductStatus - Only Org2 (Retailer) can update, and only if current owner
        is Org1MSP
47.         func (s *SmartContract) UpdateProductStatus(ctx
        contractapi.TransactionContextInterface, id, status, location string) error {
48.     mspID, err := ctx.GetClientIdentity().GetMSPID()
49.     if err != nil || mspID != "Org2MSP" {
50.         return fmt.Errorf("only Org2 (Retailer) can update product status")
51.     }
52.
53.     productJSON, err := ctx.GetStub().GetState(id)
54.     if err != nil || productJSON == nil {
55.         return fmt.Errorf("product %s does not exist", id)
56.     }
57.
58.     var product Product
59.     _ = json.Unmarshal(productJSON, &product)
60.
61.     if product.Owner != "Org1MSP" {
62.         return fmt.Errorf("product %s is not owned by Org1; cannot be updated by
        Org2", id)
63.     }
64.
65.     product.Status = status
66.     product.Location = location
67.     product.Owner = mspID // Transfer ownership to Org2
68.
69.     updatedJSON, _ := json.Marshal(product)
70.     return ctx.GetStub().PutState(id, updatedJSON)
71. }
72.
73. // PurchaseProduct - Only Org3 (Customer) can purchase, and only if owned by
        Org2MSP
74.         func (s *SmartContract) PurchaseProduct(ctx
        contractapi.TransactionContextInterface, id string) error {
75.     mspID, err := ctx.GetClientIdentity().GetMSPID()
76.     if err != nil || mspID != "Org3MSP" {
77.         return fmt.Errorf("only Org3 (Customer) can purchase products")
78.     }
79.
80.     productJSON, err := ctx.GetStub().GetState(id)
81.     if err != nil || productJSON == nil {
82.         return fmt.Errorf("product %s does not exist", id)
```

```
83.     }
84.
85.     var product Product
86.     _ = json.Unmarshal(productJSON, &product)
87.
88.     if product.Owner != "Org2MSP" {
89.       return fmt.Errorf("product %s is not owned by Org2; cannot be sold to Org3",
        id)
90.     }
91.
92.     product.Status = "SOLD"
93.     product.Owner = mspID
94.
95.     updatedJSON, _ := json.Marshal(product)
96.     return ctx.GetStub().PutState(id, updatedJSON)
97. }
98.
99. // ProductExists utility function
100.            func            (s            *SmartContract)            ProductExists(ctx
        contractapi.TransactionContextInterface, id string) (bool, error) {
101.    data, err := ctx.GetStub().GetState(id)
102.    return data != nil, err
103. }
104.
105. // GetProduct - Anyone can read product by ID
106. func (s *SmartContract) GetProduct(ctx contractapi.TransactionContextInterface,
        id string) (*Product, error) {
107.    productJSON, err := ctx.GetStub().GetState(id)
108.    if err != nil || productJSON == nil {
109.      return nil, fmt.Errorf("product %s not found", id)
110.    }
111.
112.    var product Product
113.    _ = json.Unmarshal(productJSON, &product)
114.    return &product, nil
115. }
116.
117. // GetAllProducts - Return all products (open query for simplicity)
118.            func            (s            *SmartContract)            GetAllProducts(ctx
        contractapi.TransactionContextInterface) ([]*Product, error) {
119.    iterator, err := ctx.GetStub().GetStateByRange("", "")
120.    if err != nil {
121.      return nil, err
122.    }
123.    defer iterator.Close()
124.
125.    var products []*Product
126.    for iterator.HasNext() {
127.      item, err := iterator.Next()
```

```
128.        if err != nil {
129.                return nil, err
130.        }
131.        var product Product
132.        _ = json.Unmarshal(item.Value, &product)
133.        products = append(products, &product)
134.    }
135.
136.    return products, nil
137. }
138.
139. func main() {
140.    chaincode, err := contractapi.NewChaincode(new(SmartContract))
141.    if err != nil {
142.        panic("Error creating chaincode: " + err.Error())
143.    }
144.    if err := chaincode.Start(); err != nil {
145.        panic("Error starting chaincode: " + err.Error())
146.    }
147. }
```

APPENDIX B

NETWORK SETUP AND CLI COMMANDS

This appendix provides a complete, step-by-step record of the commands used to deploy and test the Agri-food Supply Chain Management system. All commands are run from the fabric-samples/test-network directory unless specified otherwise.

Phase 1: Environment and Network Initialization

These commands prepare the environment and bring up the core three-organization network.

1. **Clean up previous runs (if any):**

   ./network.sh down

2. **Launch a two-organization network with a channel:**

   ./network.sh up createChannel -ca

3. **Add the third organization (Org3) to the network:**

   Bash

   ./addOrg3.sh up -c mychannel

   (Note: This command is run from the addOrg3 directory.)

4. **Set the environment variables for Fabric binaries:**

   export PATH=$PATH:~/agrifood/fabric-samples/bin

5. **Set the configuration path:**

   export FABRIC_CFG_PATH=$PWD/../config

Phase 2: Chaincode Installation and Deployment

This phase covers the entire lifecycle of the agrifood chaincode, from packaging to committing it on the channel.

6. **Package the chaincode:**

```
peer lifecycle chaincode package agrifood_1.tar.gz \
--path ../chaincode/agrifood \
--lang golang \
--label agrifood_1
```

7. **Set Org1's environment variables for CLI operations:**

```
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_ADDRESS=localhost:7051
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/
org1.example.com/users/Admin@org1.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizati
ons/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export FABRIC_CFG_PATH=$PWD/../config
export CORE_PEER_TLS_ENABLED=true
```

8. **Install the chaincode on Org1's peer:**

```
peer lifecycle chaincode install agrifood_1.tar.gz
```

9. **Query the installed packages to get the package ID:**

```
peer lifecycle chaincode queryinstalled
```

10. **Approve the chaincode definition for Org1:**

```
peer lifecycle chaincode approveformyorg \
--channelID mychannel \
--name agrifood \
--version 1.0 \
--package-id
agrifood_1:6e7c894ae7849b284eadcd1c58a7775cefc5602984afce378c8d1c8b
540e685f \
--sequence 1 \
--signature-policy
"OR('Org1MSP.member','Org2MSP.member','Org3MSP.member')" \
--tls \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
```

```
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

11. **Set Org2's environment variables:**

```
export CORE_PEER_LOCALMSPID="Org2MSP"
export CORE_PEER_ADDRESS=localhost:9051
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/
org2.example.com/users/Admin@org2.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizati
ons/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

12. **Approve chaincode for Org2:**

```
peer lifecycle chaincode approveformyorg \
--channelID mychannel \
--name agrifood \
--version 1.0 \
--package-id
agrifood_1:6e7c894ae7849b284eadcd1c58a7775cefc5602984afce378c8d1c8b
540e685f \
--sequence 1 \
--signature-policy
"OR('Org1MSP.member','Org2MSP.member','Org3MSP.member')" \
--tls \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

13. **Set Org3's environment variables:**

```
export CORE_PEER_LOCALMSPID="Org3MSP"
export CORE_PEER_ADDRESS=localhost:11051
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/
org3.example.com/users/Admin@org3.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizati
ons/org3.example.com/peers/peer0.org3.example.com/tls/ca.crt
```

14. **Approve chaincode for Org3:**

```
peer lifecycle chaincode approveformyorg \
--channelID mychannel \
```

```
--name agrifood \
--version 1.0 \
--package-id
agrifood_1:a5b8b5aa9bdcce4c62ab17bb439e45135377827ea2488585641ed9c
2700c5252 \
--sequence 1 \
--signature-policy
"OR('Org1MSP.member','Org2MSP.member','Org3MSP.member')" \
--tls \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

15. **Commit the chaincode definition (using Org1's environment):**

```
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_ADDRESS=localhost:7051
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/
org1.example.com/users/Admin@org1.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizati
ons/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
peer lifecycle chaincode commit \
--channelID mychannel \
--name agrifood \
--version 1.0 \
--sequence 1 \
--signature-policy
"OR('Org1MSP.member','Org2MSP.member','Org3MSP.member')" \
--tls \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.e
xample.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.e
xample.com/tls/ca.crt \
--peerAddresses localhost:11051 \
```

```
--tlsRootCertFiles
$PWD/organizations/peerOrganizations/org3.example.com/peers/peer0.org3.e
xample.com/tls/ca.crt
```

Phase 3: Testing Chaincode Functions and Access Control

These commands test the core functionality and validate the access control logic of your chaincode.

16. **Test Case 1: Create a product as Org1 (Farmer):**

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.e
xample.com/tls/ca.crt \
-c '{"function":"CreateProduct","Args":["P003", "Farm C", "Grade C"]}'
```

17. **Test Case 2: Query the created product:**

```
peer chaincode query \
-C mychannel \
-n agrifood \
-c '{"function":"GetProduct","Args":["P003"]}'
```

18. **Test Case 3: Attempt to create a product as Org2 (Retailer):**

```
export CORE_PEER_LOCALMSPID="Org2MSP"
export CORE_PEER_ADDRESS=localhost:9051
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/
org2.example.com/users/Admin@org2.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizati
ons/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
```

```
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"CreateProduct","Args":["P888", "CityMarket", "Grade B"]}'
```

19. **Test Case 4: Update product status as Org2 (Retailer):** *(Ensure Org2's environment variables are set from step 18.)*

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"UpdateProductStatus","Args":["P003", "SHIPPED",
"Warehouse"]}'
```

20. **Test Case 5: Attempt to update status as Org3 (Customer):**

```
export CORE_PEER_LOCALMSPID="Org3MSP"
export CORE_PEER_ADDRESS=localhost:11051
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/
org3.example.com/users/Admin@org3.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizati
ons/org3.example.com/peers/peer0.org3.example.com/tls/ca.crt
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.exa
mple.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:11051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"UpdateProductStatus","Args":["P003", "IN_TRANSIT",
"Org3 Warehouse"]}'
```

21. **Test Case 6: Invoke PurchaseProduct as Org3 (Customer):** *(Ensure Org3's environment variables are set from step 20.)*

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n agrifood \
--peerAddresses localhost:11051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"PurchaseProduct","Args":["P003"]}
```

APPENDIX C

HYPERLEDGER EXPLORER SETUP

This appendix details the step-by-step process for setting up and configuring Hyperledger Explorer to visualize the Agri-food Supply Chain network.

Phase 1: Explorer Environment Setup

1. **Create a new directory** named explorer and navigate into it.

   ```
   mkdir explorer
   cd explorer
   ```

2. **Copy the necessary configuration files** and the Docker Compose file from the Hyperledger Explorer repository.

   ```
   wget https://raw.githubusercontent.com/hyperledger/blockchain-explorer/main/examples/net1/config.json
   wget https://raw.githubusercontent.com/hyperledger/blockchain-explorer/main/examples/net1/connection-profile/test-network.json -P connection-profile
   wget https://raw.githubusercontent.com/hyperledger/blockchain-explorer/main/docker-compose.yaml
   ```

3. **Copy the cryptographic material** from your Fabric network to the explorer directory. This is essential for the Explorer to authenticate with the network. If you encounter permission errors, use sudo chown to fix file ownership before copying.

   ```
   # Fix file ownership (if needed)
   sudo chown -R abdullah:abdullah /home/abdullah/go/src/github.com/Agrifood/fabric-samples/test-network/organizations

   # Copy the organizations directory
   cp -r /home/abdullah/go/src/github.com/Agrifood/fabric-samples/test-network/organizations \
   /home/abdullah/explorer/
   ```

Phase 2: Configuration and Launch

1. **Edit docker-compose.yaml** to connect to your running network. You must set the external network name to fabric_test and update the volume path to your local crypto material.

   YAML

   ```
   # ...
   ```

```
networks:
  mynetwork.com:
    external: true
    name: fabric_test
# ...
services:
  explorer.mynetwork.com:
    # ...
    volumes:
      - ./config.json:/opt/explorer/app/platform/fabric/config.json
      - ./connection-profile:/opt/explorer/app/platform/fabric/connection-profile
      - /home/abdullah/go/src/github.com/Agrifood/fabric-samples/test-network/organizations:/tmp/crypto
      - walletstore:/opt/explorer/wallet
# ...
```

2. **Edit test-network.json** to specify the correct admin private key and signed certificate paths for Org1MSP.

   JSON

```
"organizations": {
   "Org1MSP": {
      "mspid": "Org1MSP",
      "adminPrivateKey": {
         "path": "/tmp/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp/keystore/YOUR_PRIVATE_KEY_HERE_sk"
      },
      "peers": ["peer0.org1.example.com"],
      "signedCert": {
         "path": "/tmp/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp/signcerts/cert.pem"
      }
   }
},
// ...
```

3. **Launch the Explorer services** after ensuring your Fabric network is running.

   docker-compose up –d

Phase 3: Verification and Debugging

1. **Access the Dashboard:** Open a web browser and navigate to http://localhost:8080.
2. **Check Logs:** If the Explorer fails to start, you can view the logs for detailed error messages.

   ```
   docker logs -f explorer.mynetwork.com
   ```

This setup provides a complete visual tool to monitor the blocks and transactions of your Agri-food supply chain, which is essential for validating your system.

APPENDIX D

COMPLEX ENGINEERING PROBLEM

International Islamic University Chittagong
Department of Computer Science and Engineering
Report on Final Year Project/Thesis as Complex Engineering Problem

The following report will justify that the work done in your project/thesis is a solution to a complex engineering problem. To do this, firstly, specify which attributes of complex engineering problem (P) are present in the problem you have formulated and solved. Secondly, specify which complex engineering activities (A) are involved when solving the problem, particularly when you required to communicate for solving the problem. Provide explanation for each P and A involved. Finally, give an overall comment justifying that you have solved a complex engineering problem in your project/thesis.

Title of the Project/Thesis:	BLOCKCHAIN IMPLEMENTATION FOR AGRICULTURAL FOOD SUPPLY CHAIN USING HYPERLEDGER FABRIC	
Members of the Group	**Name**	**ID**
	Ashraf Ali Rakib	C213090
	Miraj Mahmud	C213062
	Md Abdullah	C213042
Name of the Supervisor	Abdullahil Kafi	

Which attributes of the Complex Engineering Problems (P) are present?

Attribute of Complex Engineering Problem	**Explanation (Justify that the attribute was present and you have addressed it sufficiently)**
P1: Depth of knowledge required	Our project required a deep understanding of blockchain concepts, Hyperledger Fabric architecture, smart contract programming in Go, and cryptographic security using SHA-256. We had to integrate these technologies practically to build a real, permissioned blockchain network connecting multiple organizations.
P2: Range of conflicting requirements	We faced several trade-offs while designing the system — ensuring transparency for consumers without compromising the privacy of farmers and retailers. Managing this balance required a lot of design thinking, which we solved using Fabric's role-based access control and channel privacy features.

P3: Depth of analysis required (no obvious solution)	There was no direct or pre-existing solution for blockchain-based agricultural traceability in Bangladesh. We had to explore, compare, and analyze different blockchain frameworks before selecting Hyperledger Fabric and designing our own architecture, consensus setup, and chaincode logic.
P4: Familiarity of issues (infrequently encountered issues)	The use of blockchain in agriculture is still quite new in our country. We had to deal with unfamiliar issues such as configuring multiple organizations, handling certificate authorities, and deploying chaincode across peers — all of which required research and trial-and-error learning.
P5: Extent of applicable codes	The system involved writing and deploying smart contracts, configuring Fabric network components, and using CLI commands for blockchain operations. We worked extensively with network setup scripts, chaincode lifecycle commands, and Fabric's security policies, which are beyond traditional coding practices.
P6: Extent of stakeholder involvement and conflicting requirements	Our system was built for multiple stakeholders farmers, retailers, and consumers each with unique needs. Farmers wanted fair pricing and product authentication, retailers needed efficient record management, and consumers demanded trust and transparency. Our blockchain network brought all of them together securely.
P7: Interdependence	The project's architecture was highly interdependent. Every component peer nodes, orderers, certificate authorities, and smart contracts had to work together seamlessly. Any misconfiguration could affect the whole network, so coordination and testing were critical.

Which attributes of Complex Engineering Activities (A) are involved?

Attribute of Complex Engineering Activities	**Explanation (Justify that the attribute was present and you have addressed it sufficiently)**
A1: Range of resources	We used a wide variety of resources such as Hyperledger Fabric for blockchain development, Docker for containerization, Go language for smart contracts, and Hyperledger Explorer for visualization. Managing all these together required both technical and coordination skills.
A2: Level of interactions	Our project demanded frequent interaction among team members and our supervisor. We regularly discussed errors, design improvements, and testing results. The setup of multi-organization networks also required team collaboration for configuring different nodes.
A3: Innovation	We introduced a new blockchain-based traceability model that automatically tracks agricultural products from farm to consumer. The use of Raft consensus, along with a role-

	based permission system, made our solution more reliable and transparent than traditional centralized models.
A4: Consequences to society and the environment	The system can have a real impact on society by ensuring food authenticity, improving farmers' income, and increasing consumer trust. It also encourages fair trade and sustainable farming practices by making every step of the supply chain transparent.
A5: Familiarity	At the beginning, we were not familiar with blockchain deployment or Fabric's technical structure. We learned everything from the ground up from Docker setup to chaincode development and performance testing — which made the project a great learning experience.

Comments: This project clearly qualifies as a **Complex Engineering Problem** because it addresses a real-world issue trust and traceability in the agricultural food supply chain through an advanced technological solution. It required us to integrate multiple technologies, solve unfamiliar problems, and coordinate as a team to develop a working blockchain network. The final system demonstrates innovation, technical depth, and social impact.

Approval of Supervisor	Approval of Examination Committee
Signature	**Signature**